SIEMENS

TIA Portal Openness: API for automation of engineering workflows

Manual

20.00.00.00

01/2025

NOTICE

This document was downloaded from the following site:

https://docs.tia.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

01/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 TIA Portal Openness: API for automation of engineering workflows 14
1.1 Cybersecurity information 14
1.2 What's new in TIA Portal Openness? 14
1.3 Basics 18
1.3.1 Installation 18
1.3.1.1 Requirements for TIA Portal Openness 18
1.3.1.2 Installing TIA Portal Openness 19
1.3.1.3 Adding users to the 'Siemens TIA Openness' user group 20
1.3.1.4 Using Windows Service 25
1.3.1.5 Accessing the TIA Portal 26
1.3.1.6 Configurations 27
1.3.2 Openness tasks 29
1.3.2.1 Introduction 29
1.3.2.2 Applications 30
1.3.2.3 Export/import 31
1.4 TIA Portal Openness API 31
1.4.1 TIA Portal Openness object 31
1.4.1.1 TIA Portal Openness object model 31
1.4.1.2 Blocks and types of the TIA Portal Openness object model 36
1.4.1.3 Hierarchy of hardware objects of the object model 41
1.4.1.4 Object list 43
1.4.2 General functions 50
1.4.2.1 Introduction 50
1.4.2.2 Information about installed TIA Portal Openness versions 50
1.4.2.3 TIA Portal Openness IntelliSense support 51
1.4.2.4 Standard libraries 52
1.4.2.5 Use of the code examples 53
1.4.2.6 Example program 56
1.4.2.7 Programming steps 61
1.4.2.8 Connecting to the TIA Portal 62
1.4.2.9 TIA Portal Openness firewall 68
1.4.2.10 Event handlers 73
1.4.2.11 Program-controlled acknowledgement of dialogs with system events 75
1.4.2.12 Terminating the connection to the TIA Portal 78
1.4.2.13 Diagnostic interfaces on TIA Portal 79
1.4.2.14 Exclusive access 0
1.4.2.15 Dynamic behaviours 0
1.4.2.16 Working with associations 0
1.4.2.17 Working with compositions 0

1.4.2.18 Verifying object equality 0
1.4.2.19 Read operations for attributes 0
1.4.2.20 Transaction handling 0
1.4.2.21 Creating a DirectoryInfo/FileInfo object 0
1.4.2.22 Self-description support for attributes, navigators, actions, and services 0
1.4.2.23 Setting attributes of Blocks, DBs & UDTs 0
1.4.2.24 Notes on performance of TIA Portal Openness 0
1.4.3 Functions for projects and project data 0
1.4.3.1 Opening a project 0
1.4.3.2 Creating a project 0
1.4.3.3 Accessing general settings of the TIA Portal 0
1.4.3.4 Archiving and Retrieving a project 0
1.4.3.5 Accessing read-only TIA Portal project 0
1.4.3.6 Accessing languages 0
1.4.3.7 Determining the object structure and attributes 0
1.4.3.8 Access software target 0
1.4.3.9 Accessing and enumerating multilingual texts 0
1.4.3.10 Updating project properties 0
1.4.3.11 Read project related attributes 0
1.4.3.12 Deleting project graphics 0
1.4.3.13 Compiling a project 0
1.4.3.14 Modifying project in UMAC 0
1.4.3.15 Saving a project 0
1.4.3.16 Closing a project 0
1.4.3.17 Export/Import system diagnostics settings 0
1.4.3.18 Identifying cross session object for Hardware & Software 0
1.4.3.19 Managing exception for missing component in project 0
1.4.3.20 Accessing multilingual support 0
1.4.4 Functions for accessing Teamcenter Gateway 0
1.4.4.1 Connecting to the Teamcenter Gateway 0
1.4.4.2 Disconnecting to the Teamcenter Gateway 0
1.4.4.3 Connecting to the ConnectSSO 0
1.4.4.4 Saving Project/Global library in Teamcenter 0
1.4.4.5 Saving Project/Global library with proxy object 0
1.4.4.6 Saving Project/Global library with new item 0
1.4.4.7 Accessing custom attributes in Teamcenter 0
1.4.4.8 Saving Project to Teamcenter as new item 0
1.4.4.9 Saving project using save to item with proxy object 0
1.4.4.10 Saving Project as new item with proxy object 0
1.4.4.11 Saving project using save as new revision with proxy object 0
1.4.4.12 Saving Project to Teamcenter as new revision 0
1.4.4.13 Downloading Project/Global library from Teamcenter 0
1.4.4.14 Searching Project / Global library from Teamcenter 0
1.4.4.15 Checkout Project/Global library dataset in Teamcenter 0
1.4.4.16 Checkin Project/Global library dataset in Teamcenter 0
1.4.4.17 Cancel checkout dataset in Teamcenter 0

1.4.5 Functions for Connections 0
1.4.5.1 Configurable attributes of a port-to-port connection 0
1.4.6 Functions on libraries 0
1.4.6.1 Functions for objects and instances 0
1.4.6.2 Accessing global libraries 0
1.4.6.3 Accessing library base type 0
1.4.6.4 Accessing global library languages 0
1.4.6.5 Opening libraries 0
1.4.6.6 Comparing libraries 0
1.4.6.7 Enumerating open libraries 0
1.4.6.8 Saving and closing libraries 0
1.4.6.9 Archiving and retrieving a library 0
1.4.6.10 Creating global libraries 0
1.4.6.11 Accessing folders in a library 0
1.4.6.12 Accessing types 0
1.4.6.13 Accessing type versions 0
1.4.6.14 Accessing blocks located in library 0
1.4.6.15 Accessing instances 0
1.4.6.16 Accessing master copies 0
1.4.6.17 Create master copy from a project in library 0
1.4.6.18 Cleaning up library 0
1.4.6.19 Create an object from a master copy 0
1.4.6.20 Copying master copies 0
1.4.6.21 Creating new type from document 0
1.4.6.22 Updating type from document 0
1.4.6.23 Determining out-of-date type instances 0
1.4.6.24 Updating the project 0
1.4.6.25 Updating a library 0
1.4.6.26 Cleaning up library 0
1.4.6.27 Harmonize project from project library 0
1.4.6.28 Updating Structure during Library Update 0
1.4.6.29 Updating libraries type 0
1.4.6.30 Deleting library content 0
1.4.6.31 Setting default version of a type 0
1.4.6.32 Getting default version of a type 0
1.4.6.33 Improved types consistency state 0
1.4.6.34 Editing library type version 0
1.4.6.35 Discarding library type version 0
1.4.6.36 Managing alarm text list in PLC and Mastercopy library 0
1.4.6.37 Managing DoNotUse in Library 0
1.4.6.38 Exporting library type version as document 0
1.4.6.39 Retrieving supported export formats for library 0
1.4.6.40 Releasing library type version 0
1.4.7 Functions for accessing devices, networks and connections 0
1.4.7.1 Open the 'Devices & networks' editor 0
1.4.7.2 Querying PLC and HMI targets 0

1.4.7.3 Accessing attributes of an address object 0
1.4.7.4 Accessing the channels of a module 0
1.4.8 Functions on networks 0
1.4.8.1 Creating a subnet 0
1.4.8.2 Accessing subnets 0
1.4.8.3 Accessing internal subnets 0
1.4.8.4 Get type identifier of subnets 0
1.4.8.5 Accessing attributes of a subnet 0
1.4.8.6 Deleting a global subnet 0
1.4.8.7 Enumerate all participants of a subnet 0
1.4.8.8 Enumerate IO systems of a subnet 0
1.4.8.9 Accessing nodes 0
1.4.8.10 Accessing attributes of a node 0
1.4.8.11 Connecting a node to a subnet 0
1.4.8.12 Disconnect a node from a subnet 0
1.4.8.13 Creating an IO system 0
1.4.8.14 Accessing the attributes of an IO system 0
1.4.8.15 Connecting an IO connector to an IO system 0
1.4.8.16 Get master system or IO system of an interface 0
1.4.8.17 Get an IO Controller 0
1.4.8.18 Get an IO Connector 0
1.4.8.19 Disconnecting an IO connector from an IO system or a DP mastersystem 0
1.4.8.20 Accessing attributes of a DP mastersystem 0
1.4.8.21 Accessing attributes of PN Driver 0
1.4.8.22 Accessing attributes of a profinet io system 0
1.4.8.23 Deleting a DP master system 0
1.4.8.24 Deleting a profinet io system 0
1.4.8.25 Creating a DP master system 0
1.4.8.26 Accessing port interconnection information of port device item 0
1.4.8.27 Attributes of port inter-connection 0
1.4.8.28 Accessing the attributes of a port 0
1.4.8.29 Enumerate DP master systems of a subnet 0
1.4.8.30 Enumerate assigned IO connectors 0
1.4.8.31 Connecting a DP IO connector to a DP master system 0
1.4.8.32 Accessing AS-i profile and parameter attributes for virtual slaves 0
1.4.8.33 Accessing CM DP as DP slave and transfer area 0
1.4.8.34 Coupling 1516 isochron central and decentral Profinet 0
1.4.8.35 Openness for CP 1604/CP 1616/CP 1626 0
1.4.8.36 Openness transfer areas for PnPn coupler 0
1.4.8.37 Openness virtual modules/submodules for ET 200SP PN HF 0
1.4.8.38 Accessing domain settings 0
1.4.8.39 Accessing redundancy mode attribute 0
1.4.8.40 Creating CCDX transfer area 0
1.4.8.41 Deleting CCDX transfer area 0
1.4.8.42 Modifying CCDX transfer areas configuration 0
1.4.9 Functions on devices 0

1.4.9.1 Mandatory attributes of devices 0
1.4.9.2 Get type identifier of devices and device items 0
1.4.9.3 Set App ID on device and device Items 0
1.4.9.4 Get App ID on device and device Items 0
1.4.9.5 Remove App ID on device and device items 0
1.4.9.6 Creating a device 0
1.4.9.7 Enumerating devices 0
1.4.9.8 Accessing devices 0
1.4.9.9 Accessing the TIA Portal hardware catalog 0
1.4.9.10 Deleting a device 0
1.4.9.11 Bulk changing hardware parameters 0
1.4.10 Functions on device items 0
1.4.10.1 Mandatory attributes of device items 0
1.4.10.2 Creating and plugging a device item 0
1.4.10.3 Moving device items into another slot 0
1.4.10.4 Copying a device item 0
1.4.10.5 Setting user defined logo settings 0
1.4.10.6 Deleting a device item 0
1.4.10.7 Enumerate device items 0
1.4.10.8 Accessing device items 0
1.4.10.9 Accessing device item as interface 0
1.4.10.10 Accessing change device 0
1.4.10.11 Accessing attributes of an I/O device interface 0
1.4.10.12 Getting subnet of device item 0
1.4.10.13 Accessing attributes of IoController 0
1.4.10.14 Accessing attributes of IoConnector 0
1.4.10.15 Accessing address controller 0
1.4.10.16 Accessing addresses 0
1.4.10.17 Accessing hardware identifiers 0
1.4.10.18 Accessing hardware identifier controller 0
1.4.10.19 Accessing channels of device items 0
1.4.10.20 Accessing normalized type identifiers 0
1.4.10.21 Creating and exporting psc file 0
1.4.10.22 Connection handling for extension racks 0
1.4.10.23 Exporting/Importing telecontrol management datapoint 0
1.4.11 Functions for accessing the data of a PLC device 0
1.4.11.1 Functions for downloading data to PLC device 0
1.4.11.2 Functions for accessing PLC service 0
1.4.11.3 Blocks 0
1.4.11.4 Tags and Tag tables 0
1.4.11.5 Functions for software units 0
1.4.11.6 Functions for fail-safe unit 0
1.4.12 Functions for accessing the data of an HMI device 0
1.4.12.1 Screens 0
1.4.12.2 Cycles 0
1.4.12.3 Text lists 0

1.4.12.4 Graphic lists 0
1.4.12.5 Connections 0
1.4.12.6 Tag table 0
1.4.12.7 VB scripts 0
1.4.13 Functions for Version Control Interface 0
1.4.13.1 Accessing VCI system group in project 0
1.4.13.2 Enumerating user groups in VCI group 0
1.4.13.3 Accessing VCI Workspace with library properties 0
1.4.13.4 Enumerating VCI mapped objects in VCI workspace 0
1.4.13.5 Creating VCI user group in VCI group 0
1.4.13.6 Connecting object 0
1.4.13.7 Updating VCI group properties 0
1.4.13.8 Deleting VCI user group 0
1.4.13.9 Enumerating VCI workspaces in VCI group 0
1.4.13.10 Creating VCI workspace in VCI group 0
1.4.13.11 Updating VCI workspace properties 0
1.4.13.12 Deleting VCI workspace 0
1.4.13.13 Exporting individual object to workspace 0
1.4.13.14 Getting supported file formats 0
1.4.13.15 Getting VCI workspace mapping properties 0
1.4.13.16 Deleting mapped object 0
1.4.13.17 Synchronizing mapped object 0
1.4.13.18 Getting status of mapped object 0
1.4.13.19 Getting child status of a mapped hierarchical object 0
1.4.13.20 Accessing synchronization status of child objects 0
1.4.13.21 Creating VCI workspace with workspace language 0
1.4.13.22 Initializing VCI object status 0
1.4.13.23 Importing Simatic ML with inactive culture 0
1.4.14 Functions support for Multiuser 0
1.4.14.1 Creating project server connection 0
1.4.14.2 Adding project to server 0
1.4.14.3 Editing project server connection 0
1.4.14.4 Deleting project server connection 0
1.4.14.5 Getting project server connection from TIA Portal Setting 0
1.4.14.6 Getting project server group 0
1.4.14.7 Adding a project to server 0
1.4.14.8 Getting server projects 0
1.4.14.9 Getting available local sessions 0
1.4.14.10 Creating a local session 0
1.4.14.11 Saving a local session 0
1.4.14.12 Deleting local session from server 0
1.4.14.13 Opening local/exclusive session 0
1.4.14.14 Opening server project 0
1.4.14.15 Checking marked objects 0
1.4.14.16 Closing local session 0
1.4.14.17 Commiting server project changes 0

1.4.14.18 Getting lockstate provider 0
1.4.14.19 Checking project locked 0
1.4.14.20 Getting lock owner 0
1.4.14.21 Checking session UptoDate 0
1.4.14.22 Getting markings from local session 0
1.4.15 Functions for accessing data of the user management 0
1.4.15.1 Functions for accessing user, roles and function rights 0
1.4.15.2 Functions for UMAC Global Users and UMC Server 0
1.4.16 Functions on OPC 0
1.4.16.1 Configuring OPC UA server secure communication protocol 0
1.4.16.2 Setting OPC UA security policy 0
1.4.16.3 Accessing OPC UA reference namespace 0
1.4.16.4 Accessing OPC UA server (SIMATIC) interface 0
1.4.16.5 Accessing OPC UA server interface 0
1.4.17 Functions for SIMATIC Drive Controller 0
1.4.17.1 Accessing PLC and X142 channel properties 0
1.4.17.2 Accessing PROFIdrive Integrated properties 0
1.4.17.3 Creating SINAMICS Integrated 0
1.4.17.4 Getting address of central module (X142) 0
1.4.17.5 Getting address of drive controller telegram 0
1.4.18 Exceptions 0
1.4.18.1 Handling exceptions 0
1.4.19 F-related Openness 0
1.4.19.1 F-related Openness 0
1.4.19.2 SafetyModificationsPossible 0
1.4.19.3 UsernameForFChangeHistory 0
1.4.19.4 SafetySignatureProvider 0
1.4.19.5 SafetyAdministration 0
1.4.19.6 SafetyPrintout 0
1.5 Export/import 0
1.5.1 Overview 0
1.5.1.1 Basic principles of importing/exporting 0
1.5.1.2 Field of application for Import/Export 0
1.5.1.3 Version Specific Simatic ML Import 0
1.5.1.4 Editing the XML file 0
1.5.1.5 Exporting configuration data 0
1.5.1.6 Importing configuration data 0
1.5.2 Import/export of project data 0
1.5.2.1 Project graphics 0
1.5.2.2 Project texts 0
1.5.3 Importing/exporting data of an HMI device 0
1.5.3.1 Importing/exporting data of an HMI device 0
1.5.3.2 Data structure for import/export 0
1.5.3.3 Tag tables 0
1.5.3.4 VB scripts 0
1.5.3.5 Text lists 0

1.5.3.6 Graphic lists 0
1.5.3.7 Connections 0
1.5.3.8 Screens 0
1.5.4 Importing/exporting data of a PLC device 0
1.5.4.1 Blocks 0
1.5.4.2 Tag tables 0
1.5.5 Importing/exporting hardware data 0
1.5.5.1 AML file format 0
1.5.5.2 Pruned AML 0
1.5.5.3 Overview of the objects and parameters of the CAx import/export 0
1.5.5.4 Structure of the CAx data for importing/exporting 0
1.5.5.5 AML type identifiers 0
1.5.5.6 Export/Import of base unit Information via AML 0
1.5.5.7 Export/Import AML with extension rack connection 0
1.5.5.8 Export/import AML file with GSD/GSDML custom attributes 0
1.5.5.9 Export/Import AML file with non-GSD/GSDML custom attributes 0
1.5.5.10 Export/import AML file with pluggable port configuration 0
1.5.5.11 Export/Import AML file with IO Link 0
1.5.5.12 Connection handling for extension racks 0
1.5.5.13 Export/Import AML file with complex tag configuration 0
1.5.5.14 Export of CAx data 0
1.5.5.15 Import of CAx data 0
1.5.5.16 Export/Import of sub modules 0
1.5.5.17 Export/Import AML file in UMAC environment 0
1.5.5.18 Export/Import AML file with normalized type identifier 0
1.5.5.19 Import CAx data without logical address 0
1.5.5.20 Exceptions during import and export of CAx data 0
1.5.5.21 Round trip exchange of devices and modules 0
1.5.5.22 Export/Import topology 0
1.5.5.23 Import of device with library references 0
1.5.5.24 Export of a device object 0
1.5.5.25 Import of a device object 0
1.5.5.26 Export/Import of device with set address 0
1.5.5.27 Export/Import of device with channels 0
1.5.5.28 Export of device item objects 0
1.5.5.29 Import of device item objects 0
1.5.5.30 Export/Import of GSD/GSDML based devices and device items 0
1.5.5.31 Export/Import device configuration with virtual interface 0
1.5.5.32 Export/Import of subnets 0
1.5.5.33 Export/Import of IO-systems 0
1.5.5.34 Export/Import of multilingual comments 0
1.5.5.35 Export/Import ET200 SP/ET200 AL devices with extension rack connections 0
1.5.5.36 Export/Import of PLC tags 0
1.5.5.37 Export/Import of RH/PLC 0
1.5.5.38 Export/Import AML with customized tag and deviceitems 0
1.5.5.39 Export/Import of FailSafe PLC 0

1.5.5.40 Export/Import of Failsafe IO 0
1.5.5.41 Export/Import vendor specific attribute 0
1.5.5.42 AML attributes versus TIA Portal Openness attributes 0
1.6 Major Changes 0
1.6.1 Major changes for long-term stability in TIA Portal Openness V19 0
1.6.2 Major changes for long-term stability in TIA Portal Openness V18 0
1.6.3 Major changes for long-term stability in TIA Portal Openness V17 0
1.6.4 Major changes in TIA Portal Openness V16 0
1.6.5 Major changes in TIA Portal Openness V15.1 0
1.6.6 Major changes in TIA Portal Openness V15 0
1.6.7 Major changes in V14 SP1 0
1.6.7.1 Major changes in V14 SP1 0
1.6.7.2 Major changes in the object model 0
1.6.7.3 Changes on pilot functionality 0
1.6.7.4 Changes for export and import 0
1.6.8 Major changes in V14 0
1.6.8.1 Major changes of the object model 0
1.6.8.2 Before updating an application to TIA Portal Openness V14 0
1.6.8.3 Major string changes 0
1.6.8.4 Import of files generated with TIA Portal Openness V13 SP1 and previous 0

TIA Portal Openness: API for automation of engineering workflows 1

1.1 Cybersecurity information

Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions only form one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit

https://www.siemens.com/global/en/products/automation/topic-areas/industrial-cybersecurity.html

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under

https://new.siemens.com/global/en/products/services/cert.html

1.2 What's new in TIA Portal Openness?

Compatibility and long term stability

• Siemens.Engineering.dll libraries
Because the Siemens.Engineering.dll libraries of V17, V18 and V19 are included in the scope of delivery, applications based on V17, V18 and V19 also run in TIA Portal V20 without modification. To make use of the latest functions of V20, you must integrate the library of V20 and recompile the application.
The Siemens.Engineering.dll libraries can be found in the installation directory under "PublicAPI\

[version]\". For example, the V20 library can be found as "C:\Program Files\Siemens\Automation\Portal V20\PublicAPI\V20\Siemens.Engineering.dll".
• Exporting SimaticML files
The Siemens.Engineering.dll libraries V17, V18, V19, V20 in the TIA Portal V20 installation will create SimaticML files of engineering version V20.
• Importing SimaticML files
Each Siemens.Engineering.dll library in TIA Portal V20 installation can import SimaticML files from engineering versions V17, V18, V19 and V20.

Note

V16 Siemens.Engineering.dll is not available as part of TIA Portal V20

Note

V* refer to the installed version of the TIA Portal Openness API.

For changes to the object model see TIA Portal Openness object model for further information.

New features

The following new features and innovations are available in TIA Portal Openness V20. For more details on the various topics, refer to the relevant sections of the product documentation.

- General
  - Get all attributes by access option
  - Unique object identifiers for PLC software objects in a project
  - Stable ordering of multilingual texts in SimaticML
- Libraries
  - New attribute “DoNotUse” on library types
  - Read all attributes of a library
  - Read all attributes of a library type version
  - Export attribute “LibraryTypeGuid” for PLC blocks and PLC types in SimaticML
  - Create new library type from documents import
  - Create new library type version from documents import
  - Export a library type as documents
  - Release a library type as new version
  - Handle conflicts when inserting master copies
  - Handle conflicts when instantiating library types
- Hardware configuration
  - Uniformed port label without whitespace

  - Find device items by classification
  - Find a device item by name in an association
  - Find a device item by position number in an association
  - PLC access control configuration for central user management (UMC)
  - PLC account locking at runtime (UMAC)
  - Access to PROFINET (sub)slot number for device item
- Hardware modules
  - Hardware parameter access for SIMATIC ET 200pro TM
  - Extended hardware parameter access for SCALANCE XC-200 (>= V.4.2), XP-200 (>= V4.3), SC-600 (>= V2.3), S615 (>= V7.1), XM-400 (>= V6.2)
  - Hardware parameter access for S7-1500 CM/CP including IRC/Telecontrol
  - Hardware parameter access for SIMATIC ET 200clean
  - Encrypted PSC file export with password for S7-1500 Software Controller V40.0
- CAx data exchange
  - Filter possible values for hardware parameters in CAx-Hardware publication tool (preview feature)
  - Support GSD and GSDX files in CAx-GSDML publication tool
- Online scenarios
  - Download View of Things (VoT) applications to PLC
- PLC user programs
  - New text-based file exchange format for PLC blocks for programming language LAD and Safety-LAD
  - Browse PLC documents (named value types)
  - Export PLC blocks, technology objects, PLC types and PLC documents (named value types) as documents
  - Import PLC blocks, technology objects, PLC types and PLC documents (named value types) from documents
  - Rename PLC tags, PLC tag tables, and PLC tag table folders
- UMAC
  - Browse device function rights
  - Delete device function rights
  - Manage UMC users and user groups
  - Create UMC users and user groups without UMC connection
- T-CPU / Technology object
  - New interface methods for using PROFIdrive Telegram 750

- Startdrive / SINAMICS DCC
  - Activate/Deactivate “functions in use” for 3rd generation drives (EPOS / PID)
  - Read all parameters on drives
  - Get telegram size in bytes
  - Get drive object number
  - Support Edge telegram
- WinCC Unified
  - Configure further dynamizations, expressions for dynamizations
  - Configure further HMI runtime settings
  - Support DB name multiplexing on HMI tags
  - Import/Export of textlists (without formatted text)
- Test Suite Advanced
  - Rename, delete and open editor for system test cases
  - Access, manage and run application test sets
- Security
  - Syslog configuration on project level
- Version Control Interface (VCI)
  - Support of following VCI workflows for the new Simatic Source Document format:

- Connect object to workspace files
- Synchronize changes (Export/Import)
- Get Status
- Update Status
- First time Export from VCI workspace

- TIA Add-Ins
  - Icon support for context menu action items
  - Create TIA Portal UI language aware Add-Ins.
- SiVArc
  - Advanced tags rules
  - SiVArc expression resolver
- Multiuser
  - Mark and unmark objects via marking service
  - Access groups on TIA Project-Server
- Web server

- Configure new system web pages for S7-1500 Hardware Controller V4.0, S7-1500 Virtual Controller V2.0 and Software Controller V40.0

See also

Major changes in TIA Portal Openness V16
Major changes for long-term stability in TIA Portal Openness V17
Major changes in TIA Portal Openness V15.1
Major changes in TIA Portal Openness V15
Major changes in V14 SP1
Major changes in V14

1.3 Basics

1.3.1 Installation

1.3.1.1 Requirements for TIA Portal Openness

Requirements for using TIA Openness applications

- A product based on the TIA Portal is installed on the PC, for example, "STEP 7 Professional" or "WinCC Professional".
- The "TIA Portal Openness" is installed on the PC.

  See Installing TIA Portal Openness

Supported Windows operating systems

The following table shows which combinations of Windows operating system, TIA Portal and user application are mutually compatible:

Windows operating system	TIA Portal	User application
64-bit	64-bit	32-bit, 64-bit and "Any CPU"

Requirements for programming TIA Portal Openness applications

- Microsoft Visual Studio 2017 or later with .Net Framework SDK 4.8 and the Windows Classic Desktop package

Necessary user knowledge

- Knowledge as a system engineer
- Advanced knowledge of Microsoft Visual Studio 2017 or later with .Net Framework SDK 4.8

- Advanced knowledge of C# / VB.net and .Net Framework
- User knowledge of the TIA Portal

TIA Portal Openness remoting channels

The TIA Portal Openness remoting channels are registered as type IpcChannel with the “ensureSecurity” parameter set to "false".

Note

You should avoid registering another IpcChannel using a “ensureSecurity” parameter value other than "false" with a priority higher than or equal to “1”.

The IpcChannel is defined with the following attributes:

Attribute	Settings
"name" and "portName"	Set to $”{Process.Name}_{Process.Id}” or $”{Process.Name}_{Process.Id}_{AppDomain.Id}” when registered in an AppDomain other than the application’s default.
“priority”	Set with the default value of “1”.
“typeFilterLevel”	Set to “Full”.
“authorizedGroup”	Set to the NTAccount value string for the built-in user account (i.e. everyone).

See also

Adding users to the "Siemens TIA Openness" user group

1.3.1.2 Installing TIA Portal Openness

Introduction

The "TIA Portal Openness" is installed via TIA Portal setup program by selecting the TIA Portal Openness checkbox (under Options) during TIA Portal installation.

Requirements

- Hardware and software of the programming device or PC meet the system requirements.
- You have administrator rights.

- Running programs are closed.
- Autorun is disabled.
- WinCC and/or STEP 7 are installed.
- The version number of the "TIA Portal Openness" matches the version numbers of WinCC and STEP 7.

Note

If a previous version of TIA Portal Openness is already installed, the current version will be installed side by side.

Procedure

To install the TIA Portal Openness, ensure the TIA Portal Openness checkbox is selected during the installation of TIA Portal. Follow the below steps to check the TIA Openness installation.

1. Under Configuration menu, select the folder Options.
2. Check the TIA Portal Openness checkbox.
3. Click "Next" and select the required option.

Follow the installation procedure of TIA Portal to complete the TIA Portal Openness installation.

Result

"TIA Portal Openness" is installed on the PC. Moreover, the local user group "Siemens TIA Openness" is generated.

Note

You still do not have access to the TIA Portal with the "TIA Portal Openness" add-on package. You need to be a member of the "Siemens TIA Openness" user group (see Adding users to the "Siemens TIA Openness" user group).

1.3.1.3 Adding users to the 'Siemens TIA Openness' user group

Introduction

When you install TIA Portal Openness on the PC, the "Siemens TIA Openness" user group is automatically created.

Whenever you access the TIA Portal with your TIA Portal Openness application, the TIA Portal verifies that you are a member of the "Siemens TIA Openness" user group, either directly or indirectly by way of another user group. If you are a member of the "Siemens TIA Openness" user group, the TIA Portal Openness application starts and establishes a connection to the TIA Portal.

Procedure

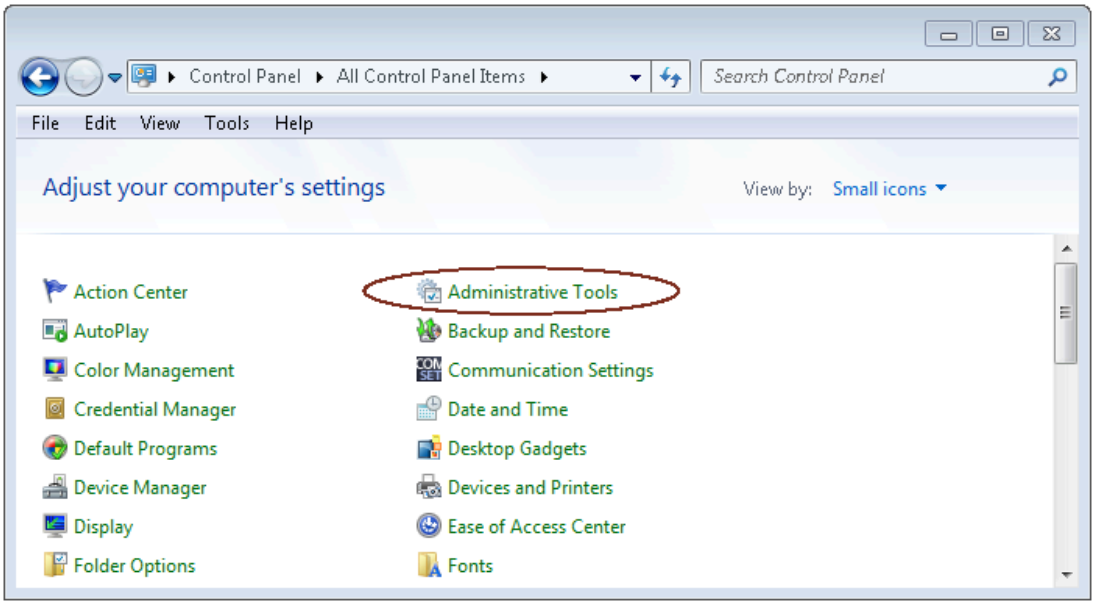
You add a user to the "Siemens TIA Openness" user group with applications from your operating system. The TIA Portal does not support this operation.

Note

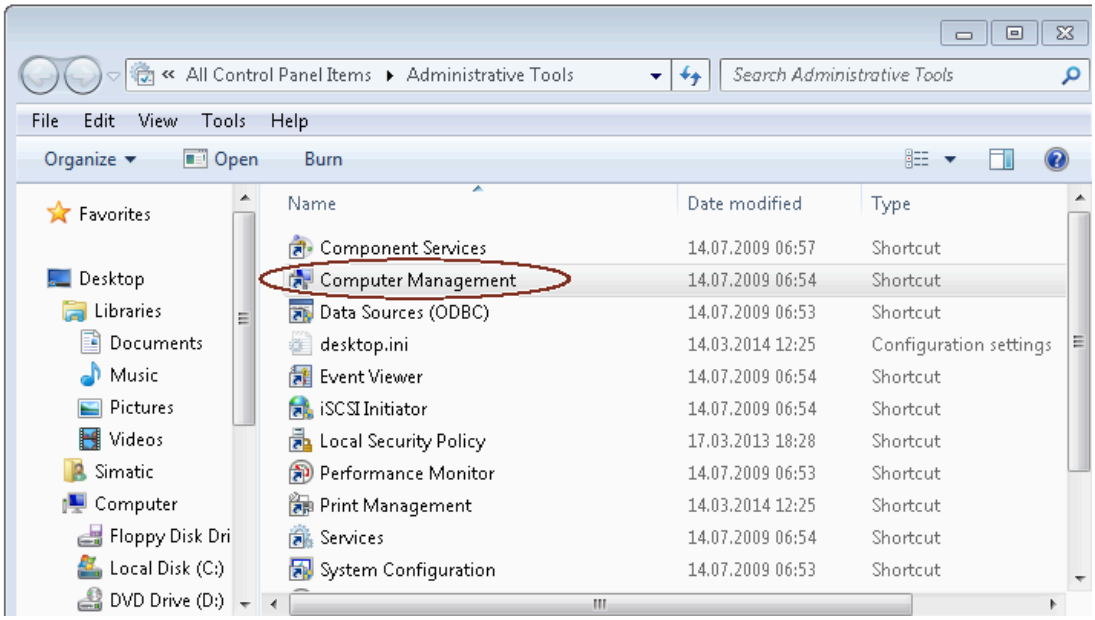
Depending on the configuration of your domain or computer, you may need to log on with administrator rights to expand the user group.

In a Windows 7 operating system (English language setting), for example, you can add a user to the user group as follows:

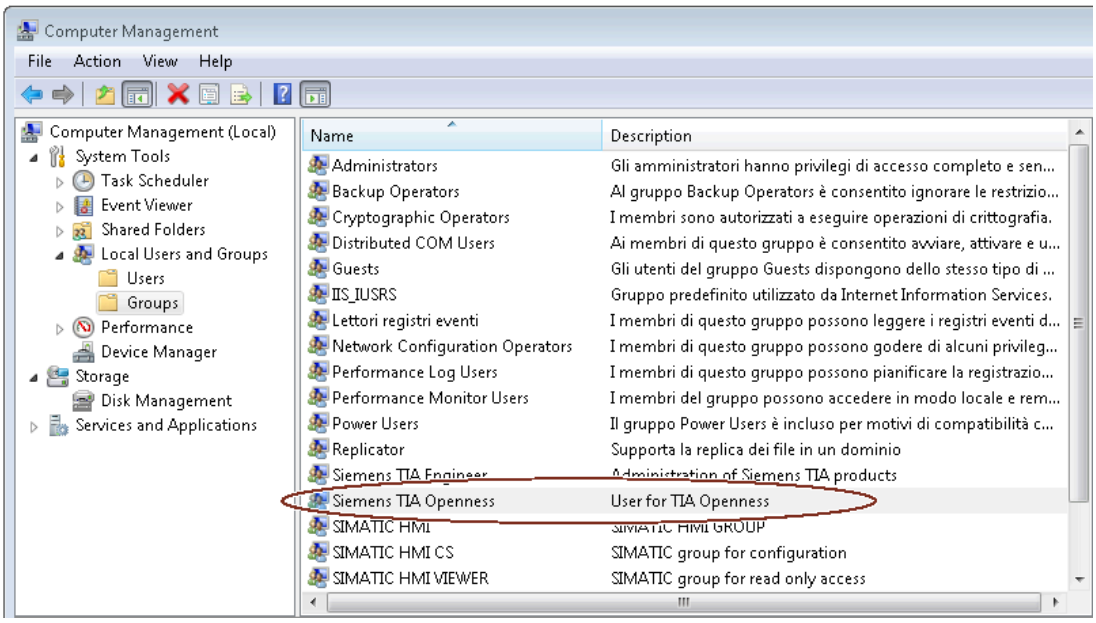
1. Select "Start" > "Control Panel".
2. Double-click "Administrative Tools" in the Control Panel.



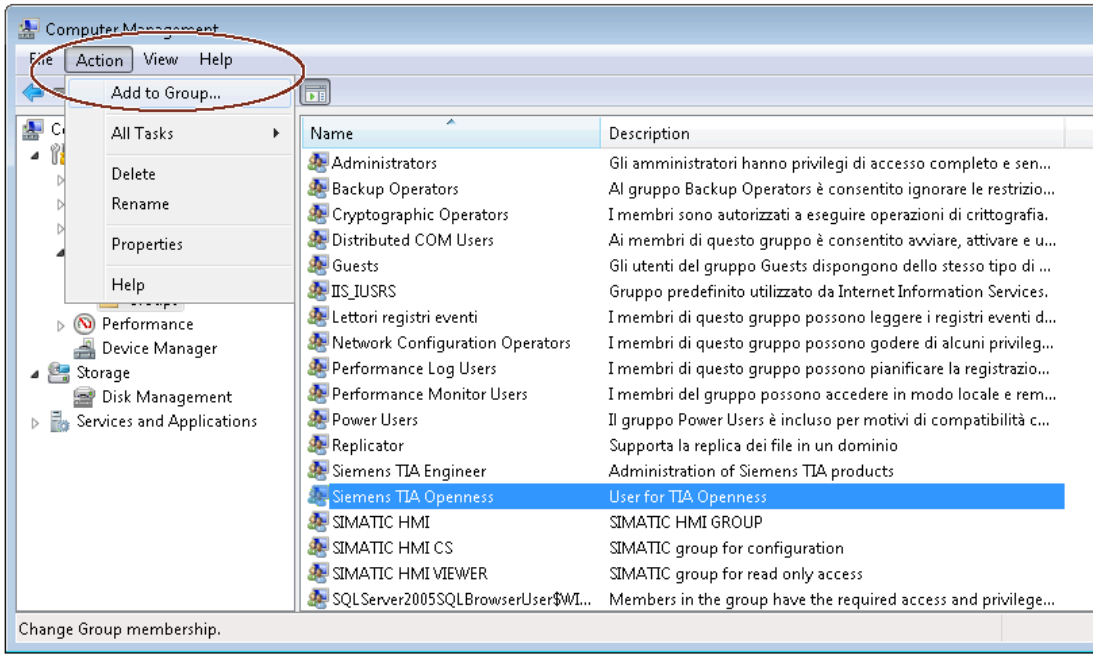
3. Click "Computer Management" to open the configuration dialog of the same name.



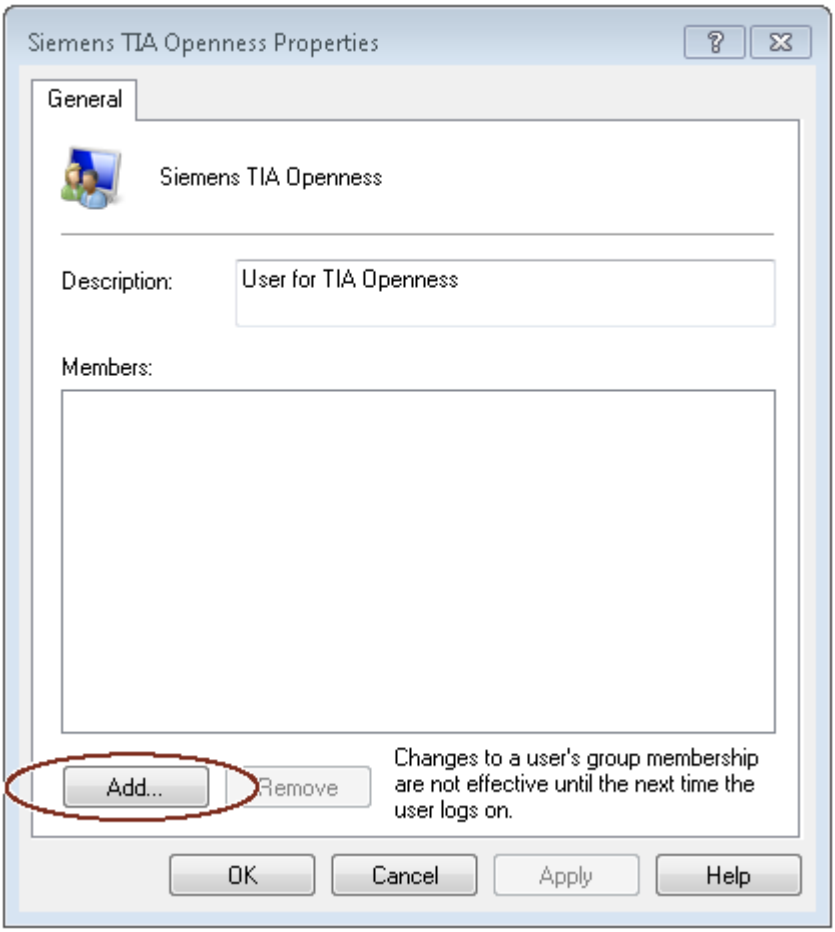
4. Select "Local Users and Groups > Groups", in order to display all created user groups.
5. Select the "Siemens TIA Openness" entry from the list of user groups in the right pane.



6. Select the "Action > Add to Group..." menu command.

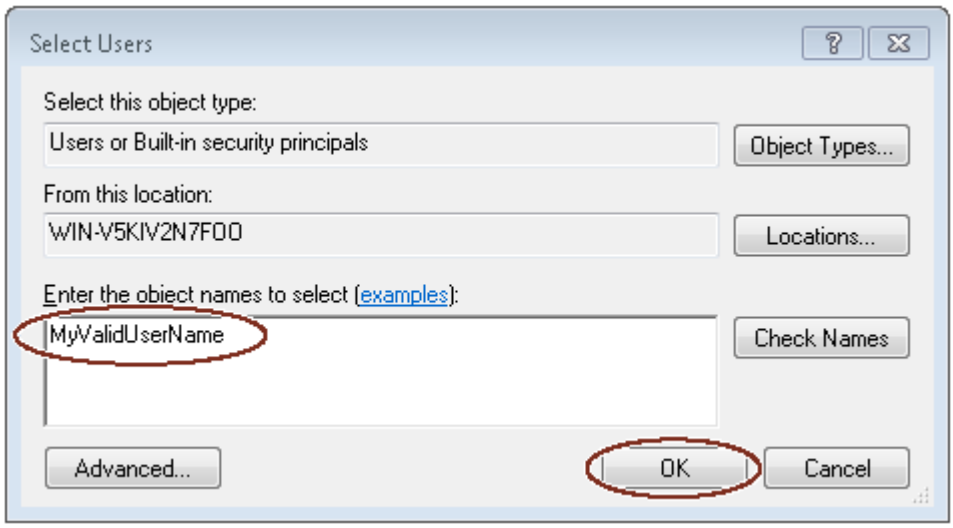


The attributes dialog of the user group opens:



7. Click "Add".

   The selection dialog that opens displays the users that can be selected:



8. Enter a valid user name in the input field.

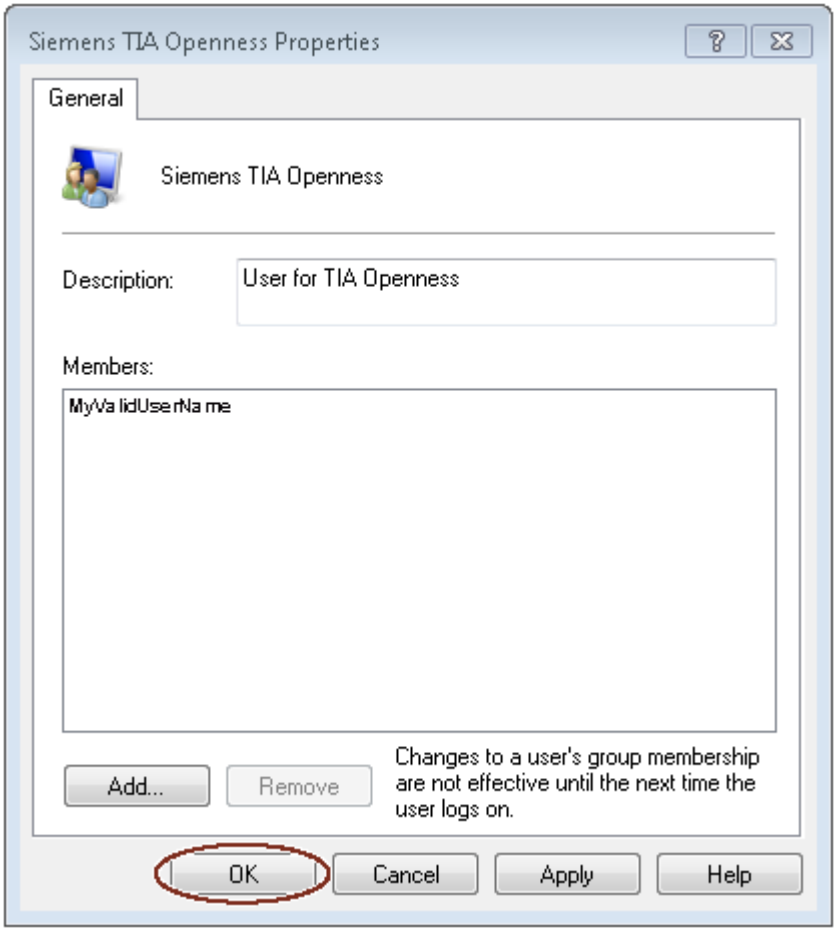
Note

Click "Check Names" to verify that the user entered has a valid user account for this domain or computer.

The "From this location" field displays the domain or computer name for the user name entered. For more information, contact your system administrator.

9. Confirm your selection with "OK".

The new user is now displayed in the attributes dialog of the user group.



You register additional users by clicking the "Add" button.

10. Click "OK" to end this operation.
11. Log on to the PC again for the changes to take effect.

1.3.1.4 Using Windows Service

Introduction

Windows Services are long-running executables that run without a user interface and are installed using an installation utility. It is possible to develop such a Windows service application that uses TIA Portal Openness.

To do so, it is necessary to create a project of type "Windows Service (.NET Framework)" in Microsoft Visual Studio.

Requirement

- The process executable of the Windows Service application needs to be whitelisted in the TIA Portal Openness Firewall beforehand. This can be done by creating the Windows registry entries on installation of the Windows Service, for example as part of a setup created with a Windows Installer project.
  For more information about whitelisting, see section "Addition of a whitelist entry without using the TIA Portal" in chapter TIA Portal Openness firewall

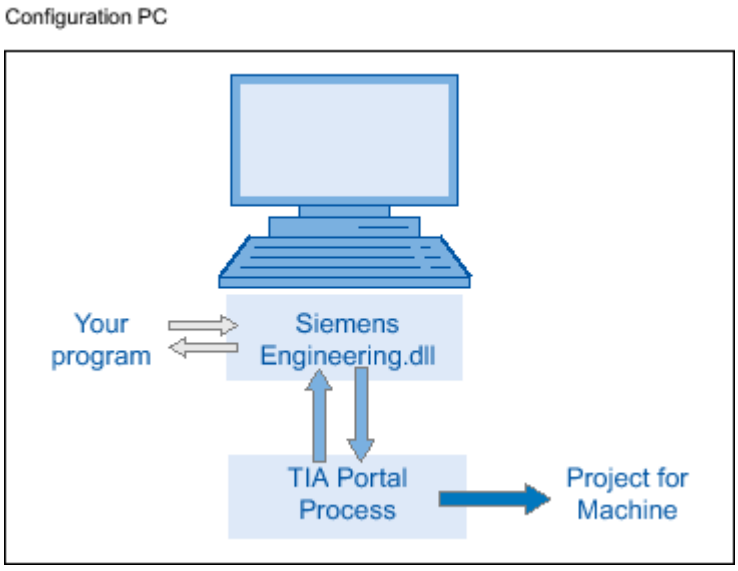
- It is not possible to access running TIA Portal processes or to start TIA Portal with graphical interface. Only a new TIA Portal instance without user interface can be created.
- It is highly recommended to run the Windows Service with credentials of a dedicated Windows user account. It is possible but not recommended to use the local service account or the network service account. The local system account is not supported due to security restrictions and will always throw a EngineeringSecurityException.
- The used Windows account (either service or user) needs to be part of the local Windows user group "Siemens TIA Openness".

See also

TIA Portal Openness firewall

1.3.1.5 Accessing the TIA Portal

Overview



Procedure

1. Set up the development environment to access and start the TIA Portal.
2. Instantiate the object of the portal application in your program to start the portal.
3. Find the desired project and open it.
4. Access the project data.
5. Close the project and exit the TIA Portal.

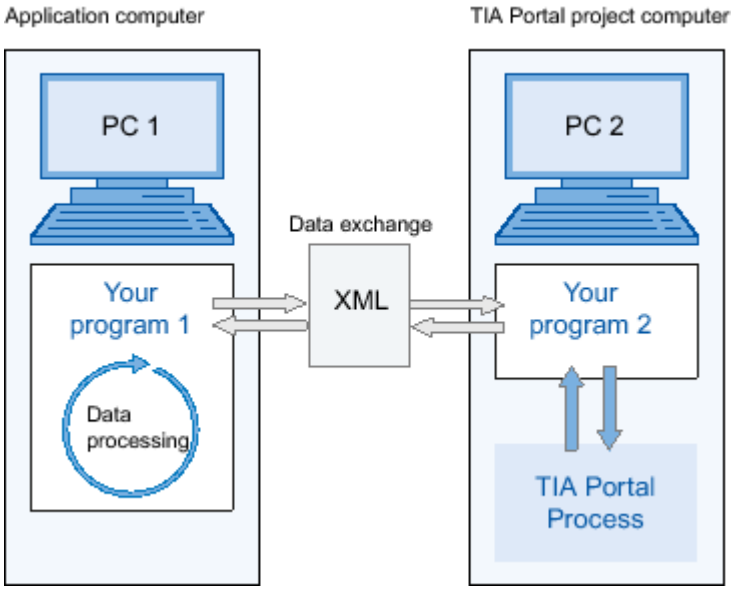
See also

Connecting to the TIA Portal
Terminating the connection to the TIA Portal

1.3.1.6 Configurations

You can work with two variants of the TIA Portal Openness:

Application and the TIA Portal are on different computers



- Data exchange takes place by XML files. The XML files can be exported or imported by your programs.
- The data exported from the TIA Portal project to PC2 can be modified on PC1 and re-imported.

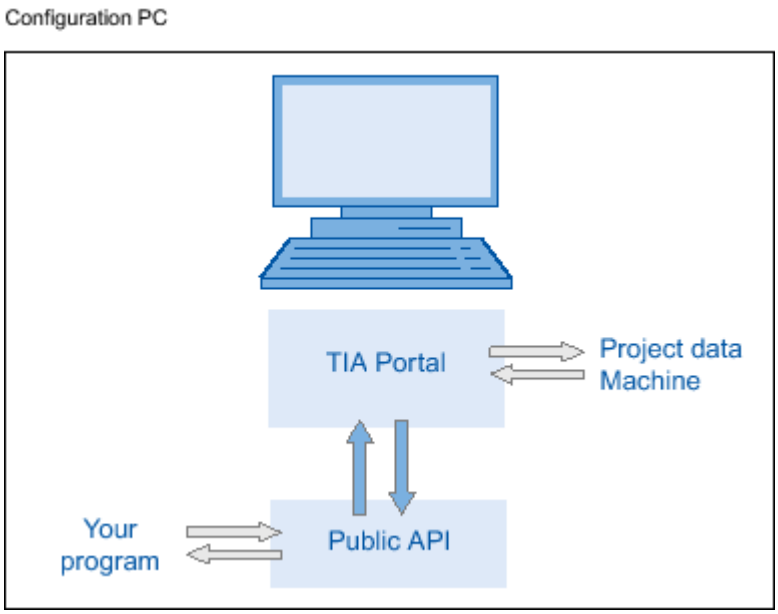
  Note

  You have to develop an executable program "Your Program 2" for PC2, such as "program2.exe". The TIA Portal runs with this program in the background.

  Import and export of XML files takes place exclusively via the TIA Portal Openness API.

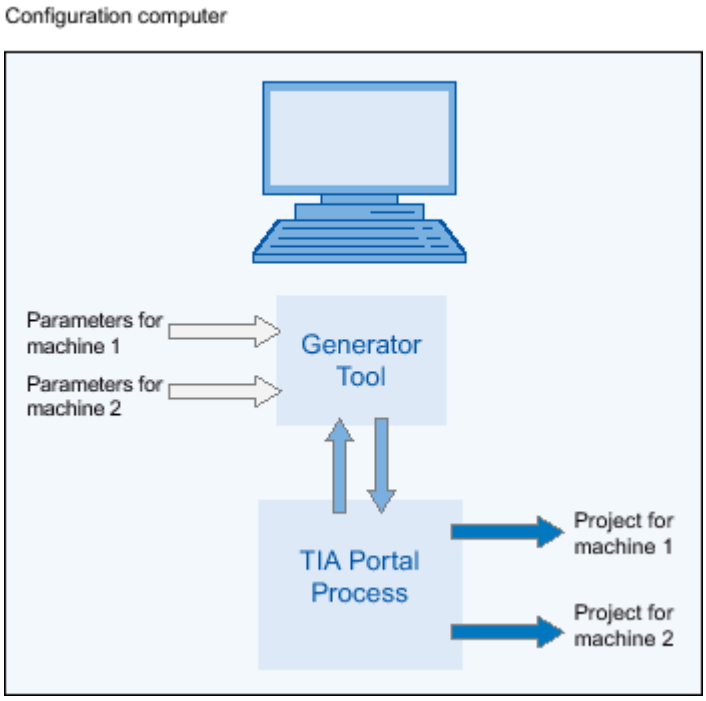
- You can archive exchanged files for verification purposes.
- Exchanged data can be processed at different locations and times.

Application and the TIA Portal are on the same computer



- Your program launches the TIA Portal either with or without the user interface. Your program opens, saves and/or closes a project. The program can also connect to a running TIA Portal.
- You can then use the TIA Portal functionality to request, generate and modify project data or to initiate import or export processes.
- The data is created under control of the TIA Portal processing and stored in the project data.

Typical application in modular mechanical engineering



- An efficient automation system is to be applied to similar machines.

- A project is available in the TIA Portal, which contains the components of all machine variants.
- The Generator tool controls the creation of the project for a specific machine variant.
- The Generator tool obtains the defaults by reading in the parameters for the requested machine variant.
- The Generator tool filters out the relevant elements from the overall TIA Portal project, modifies them if necessary and generates the requested machine project.

1.3.2 Openness tasks

1.3.2.1 Introduction

Introduction

TIA Portal Openness describes open interfaces for engineering with the TIA Portal. You will find further information about "TIA Portal Openness - Efficient generation of program code using code generators" in the SIEMENS YouTube channel.

You automate the engineering with TIA Portal Openness by controlling the TIA Portal externally from a program you have created.

You can perform the following actions with TIA Portal Openness:

- Create project data
- Modify projects and project data
- Delete project data
- Read in project data
- Make projects and project data available for other applications.

Note

Siemens is not liable for and does not guarantee the compatibility of the data and information transported via these interfaces with third-party software.

We expressly point out that improper use of the interfaces can result in data loss or production downtimes.

Note

The code snippets contained in this documentation are written in C# syntax.

Due to the shortness of the used code snippets the error handling description has been shortened as well.

Application

The TIA Portal Openness interface is used to do the following:

- Provide project data.
- Access the TIA Portal process.
- Use project data.

Using defaults from the field of automation engineering

- By importing externally generated data
- By remote control of the TIA Portal for generating projects

Providing project data of the TIA Portal for external applications

- By exporting project data

Ensuring competitive advantages through efficient engineering

- You do not have to configure existing engineering data in the TIA Portal.
- Automated engineering processes replace manual engineering.
- Low engineering costs strengthen the bidding position as compared to the competition.

Working together on project data

- Test routines and bulk data processing can take place in parallel to the ongoing configuration.

See also

Configurations

1.3.2.2 Applications

Introduction

TIA Portal Openness provides you with various ways to access the TIA Portal and offers a selection of functions for defined tasks.

You access the following areas of the TIA Portal by using the TIA Portal Openness API interface :

- Portal data
- Project data
- PLC data
- HMI data
- Drive data

Accessing the TIA Portal

TIA Portal Openness offers various ways to access the TIA Portal. You create an external TIA Portal instance in the process either with or without UI. You can also access ongoing TIA Portal processes at the same time.

Accessing projects and project data

When accessing projects and project data, you mainly use TIA Portal Openness for the following tasks:

- Close, open and save the project
- Enumerate and query objects
- Create objects
- Delete objects

1.3.2.3 Export/import

Introduction

TIA Portal Openness supports the import and export of project data by means of XML files. The import/export function supports external configuration of existing engineering data. You use this function to make the engineering process effective and free of error.

Application

You use the import/export function for the following purposes:

- Data exchange
- Copying parts of a project
- External processing of configuration data, for example, for bulk data operations using find and replace
- External processing of configuration data for new projects based on existing configurations
- Importing externally-created configuration data, for example, text lists and tags
- Providing project data for external applications

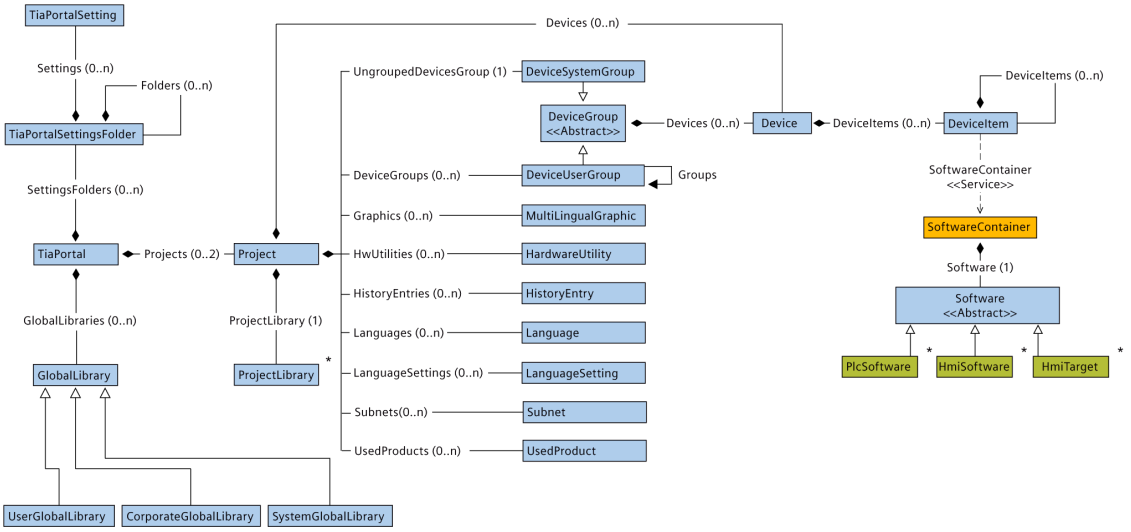
1.4 TIA Portal Openness API

1.4.1 TIA Portal Openness object

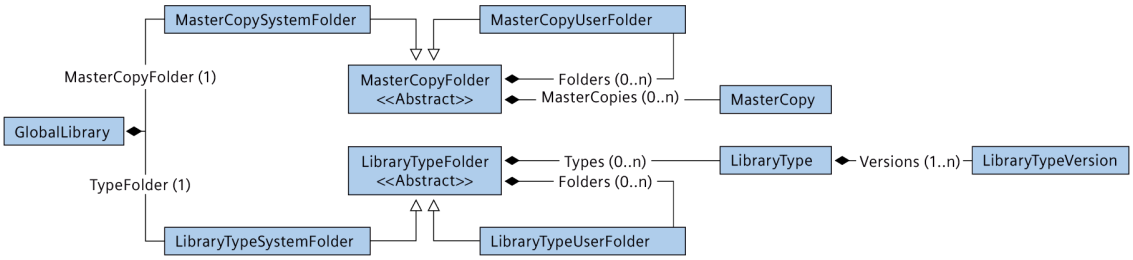
1.4.1.1 TIA Portal Openness object model

Overview

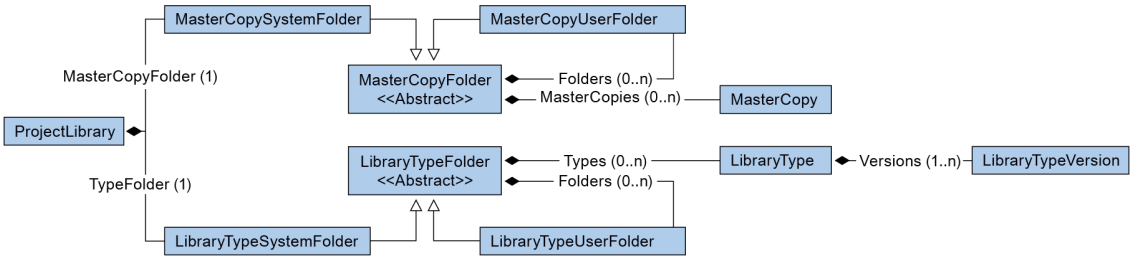
The following diagram describes the highest level of the TIA Portal Openness object model:



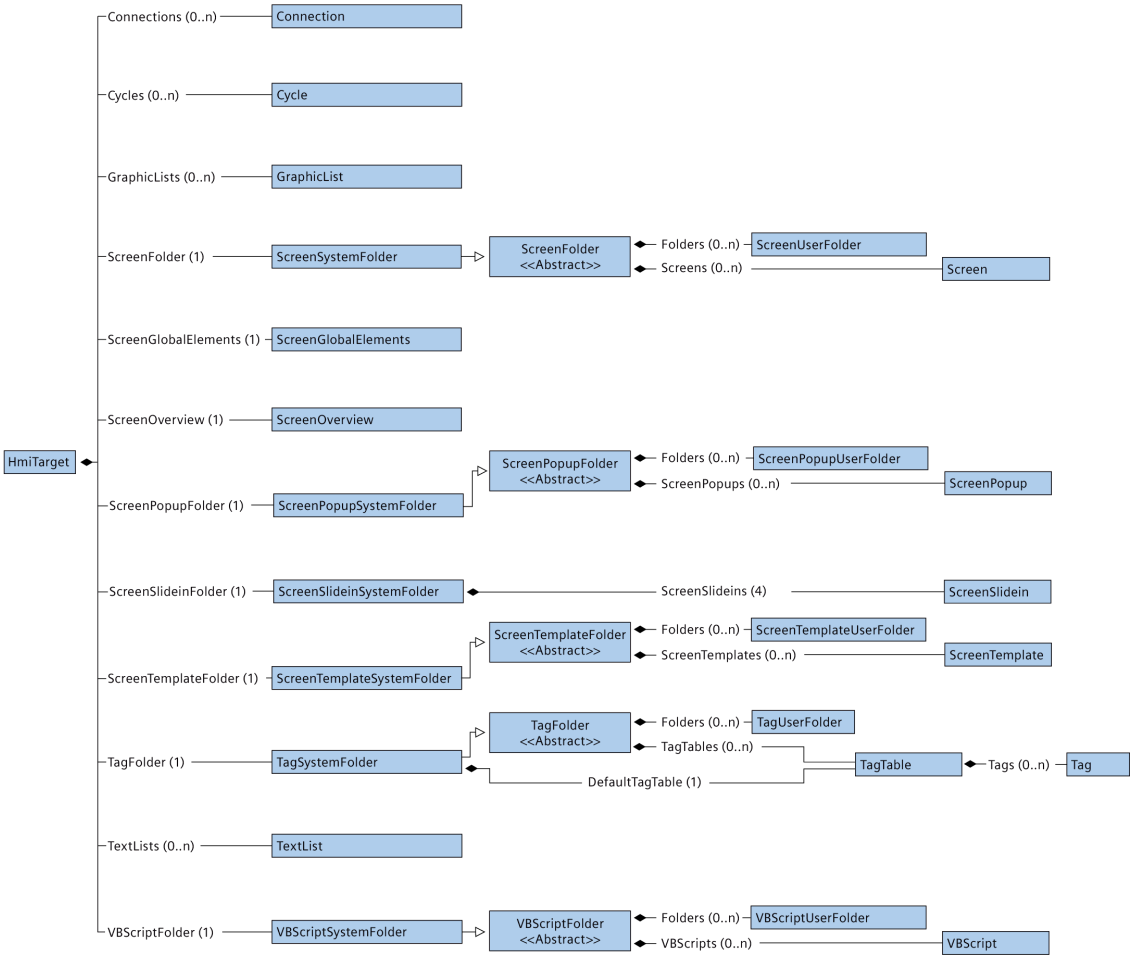
The following diagram describes the objects which are located under GlobalLibrary.



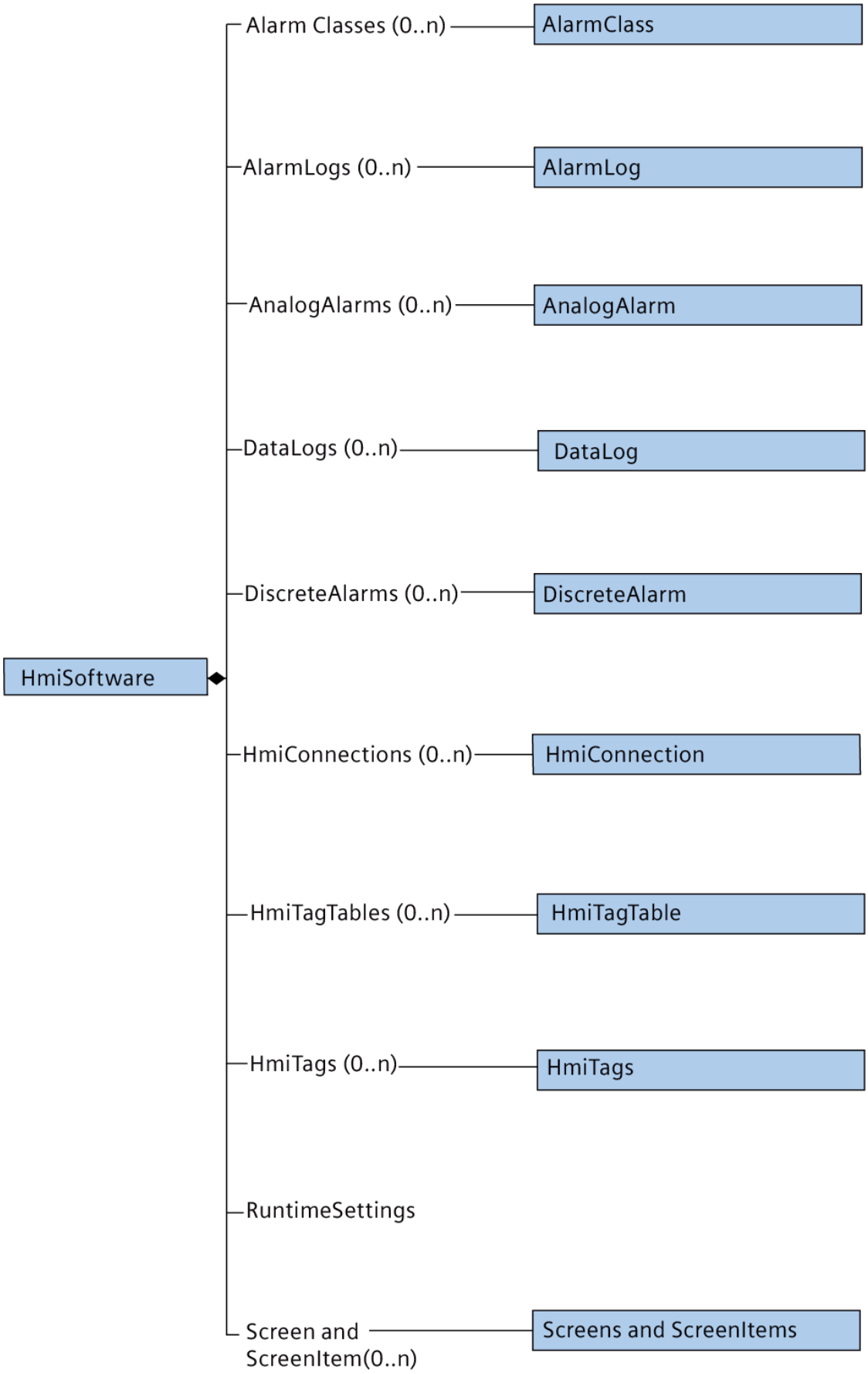
The following diagram describes the objects which are located under ProjectLibrary.



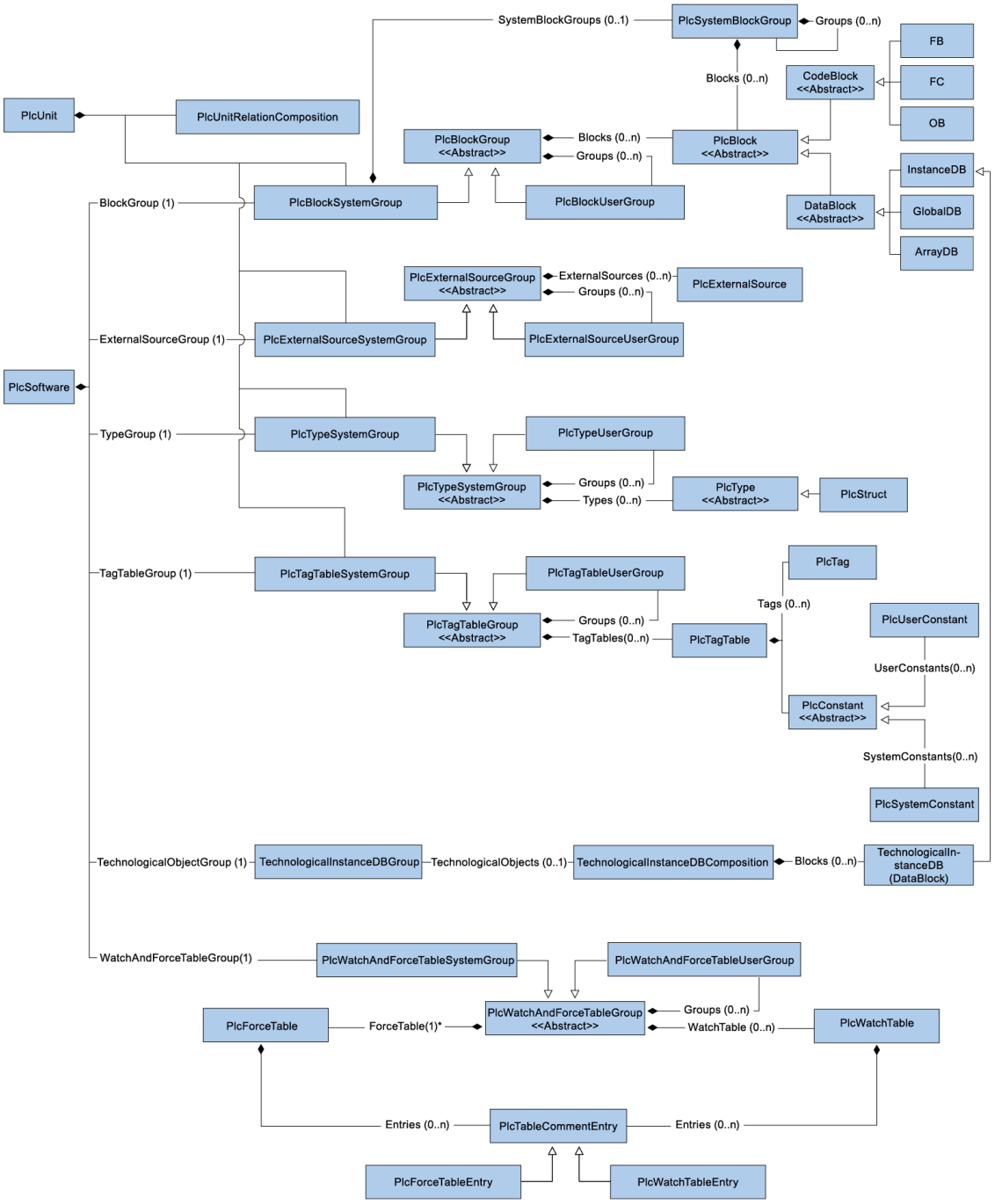
The following diagram describes the objects which are located under HmiTarget.



The following diagram describes the object which are located under HmiSoftware.



The following diagram describes the objects which are located under PlcSoftware.



Note

*The Force Table must be in PlcWatch and ForceTableSystemGroup

Access to objects in lists

You have the following options for addressing an object in a list:

- Address via the index. The counting within the lists starts with `0`.

- Use `Find` method.
  Use this method to address an object via its name. You can use this method for an composition or list. The `Find` method is not recursive.

  Example:

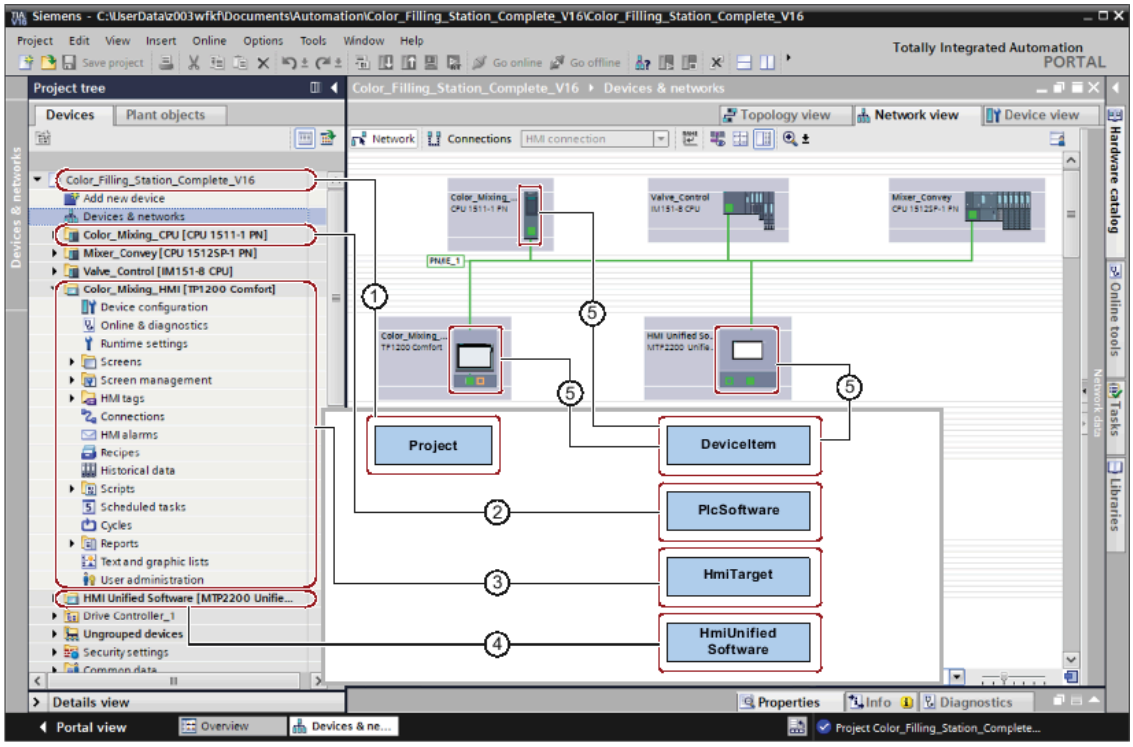
  ```
  ScreenComposition screens = folder.Screens;

  Screen screen = screens.Find("myScreen");
  ```

- Use symbolic names.

Relationship between TIA Portal and TIA Portal Openness object model

The figure below shows the relationship between the object model and a project in the TIA Portal:



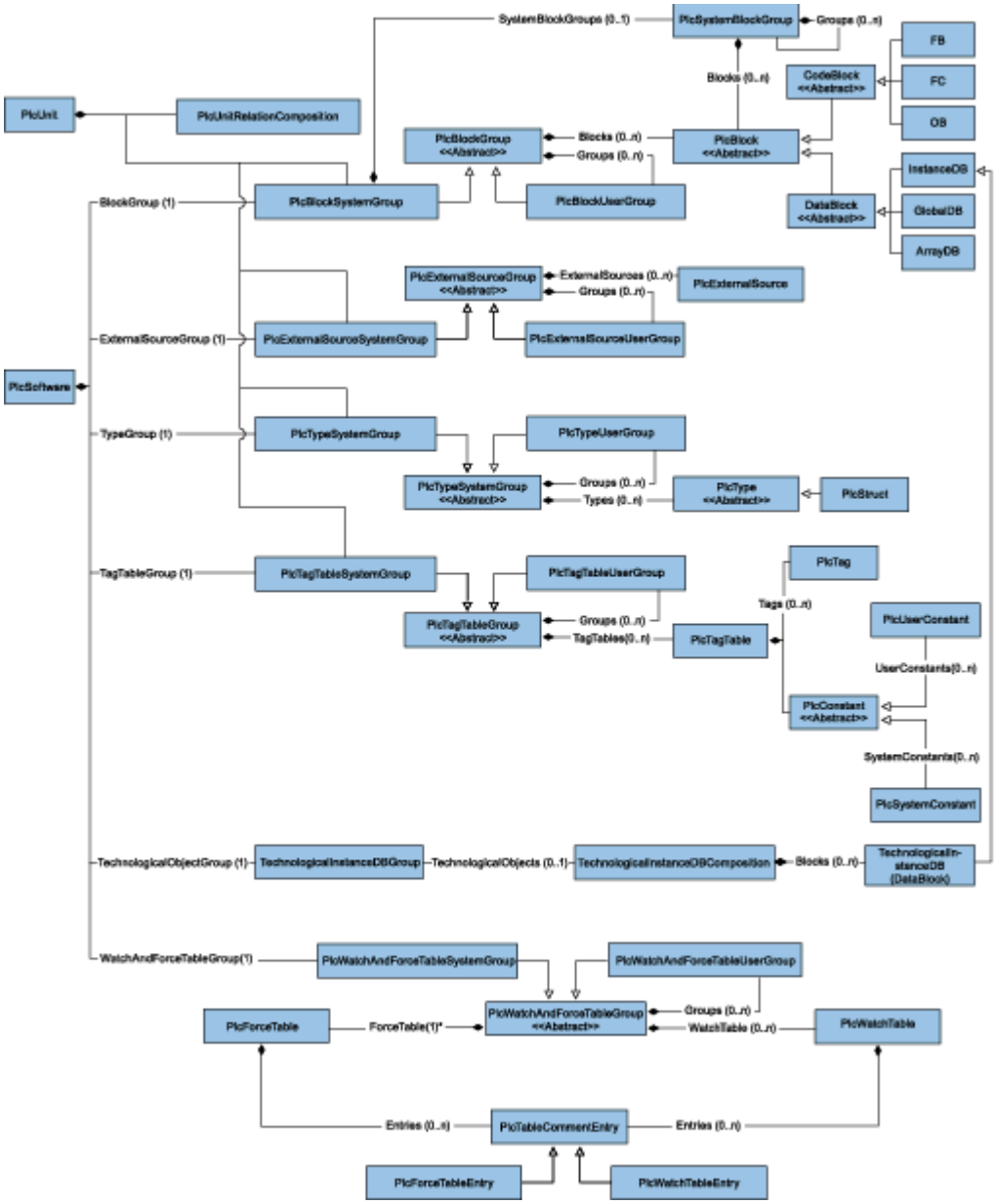
See also

Blocks and types of the TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.4.1.2 Blocks and types of the TIA Portal Openness object model

Introduction

The following diagram describes the domain model of the PLCs to give an overview of the current modeling in TIA Portal Openness.



Note

* The Force Table must be in PlcWatch and ForceTableSystemGroup

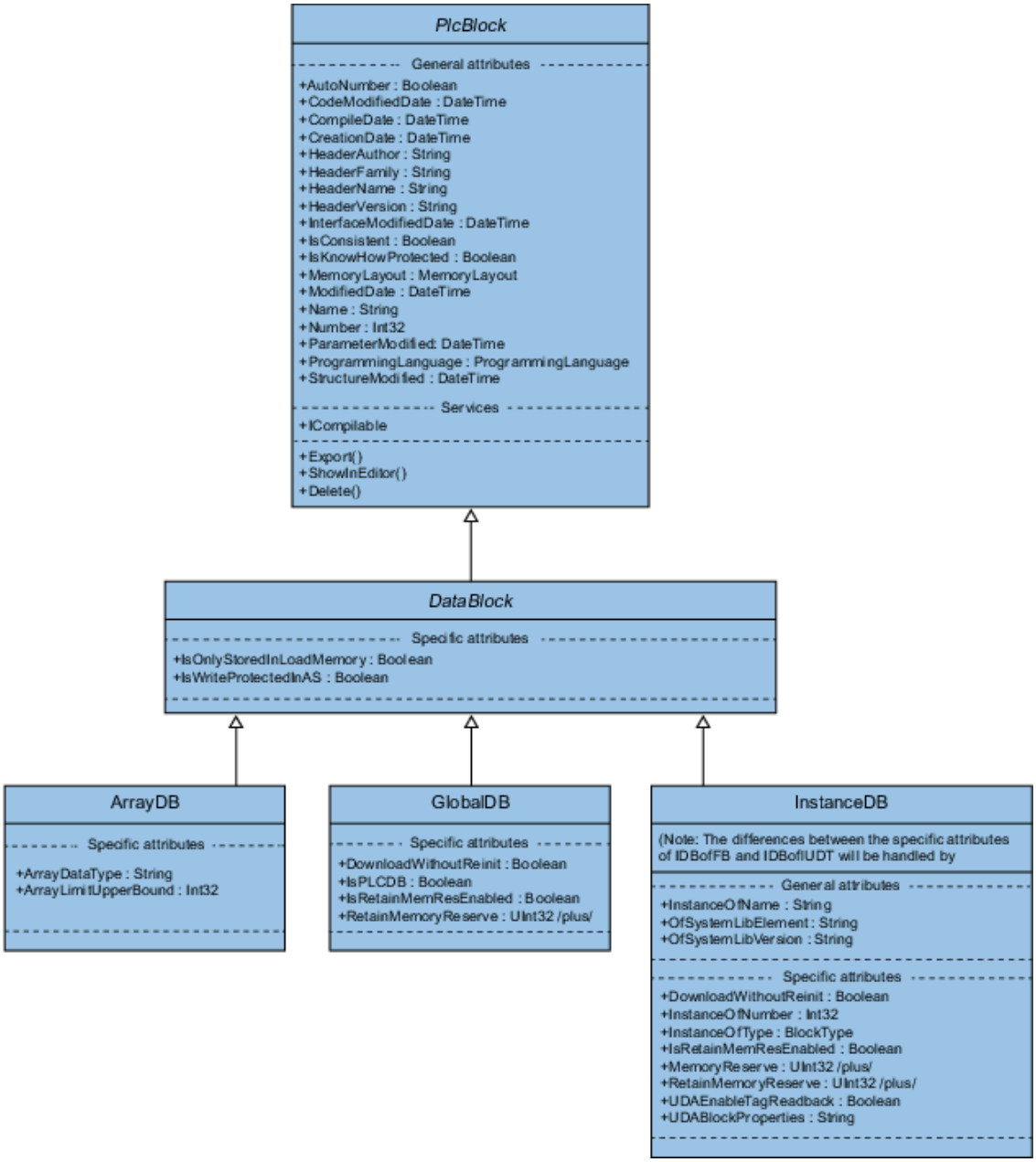
Representation of blocks and types in the TIA Portal Openness API

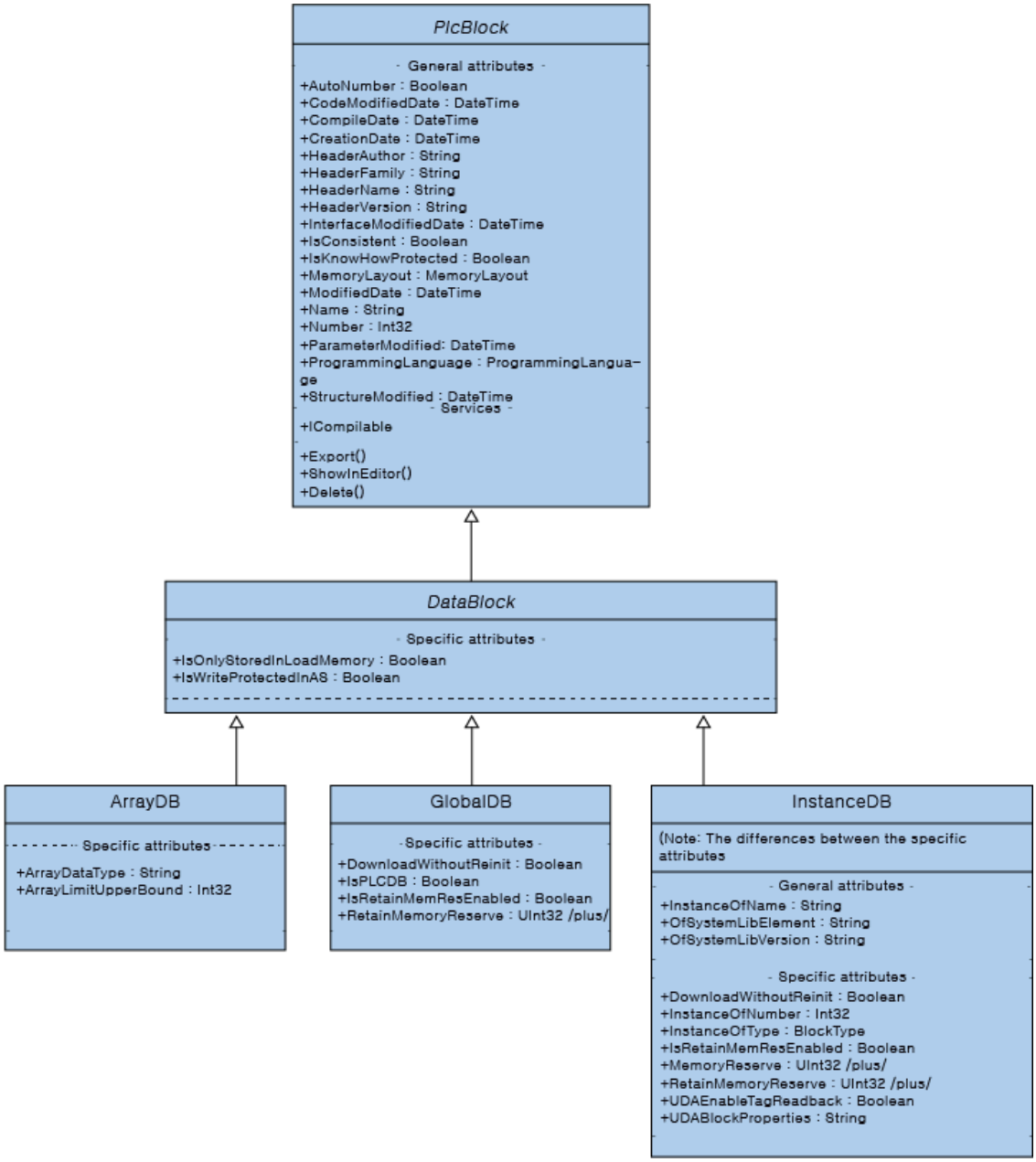
The simplified model part of blocks and for the structure is based on the attributes in the TIA Portal Openness API. These classes provide the export function and for blocks also the compile function.

Class diagrams

In the TIA Portal Openness object model all classes are defined as abstract which aren't directly instantiated.

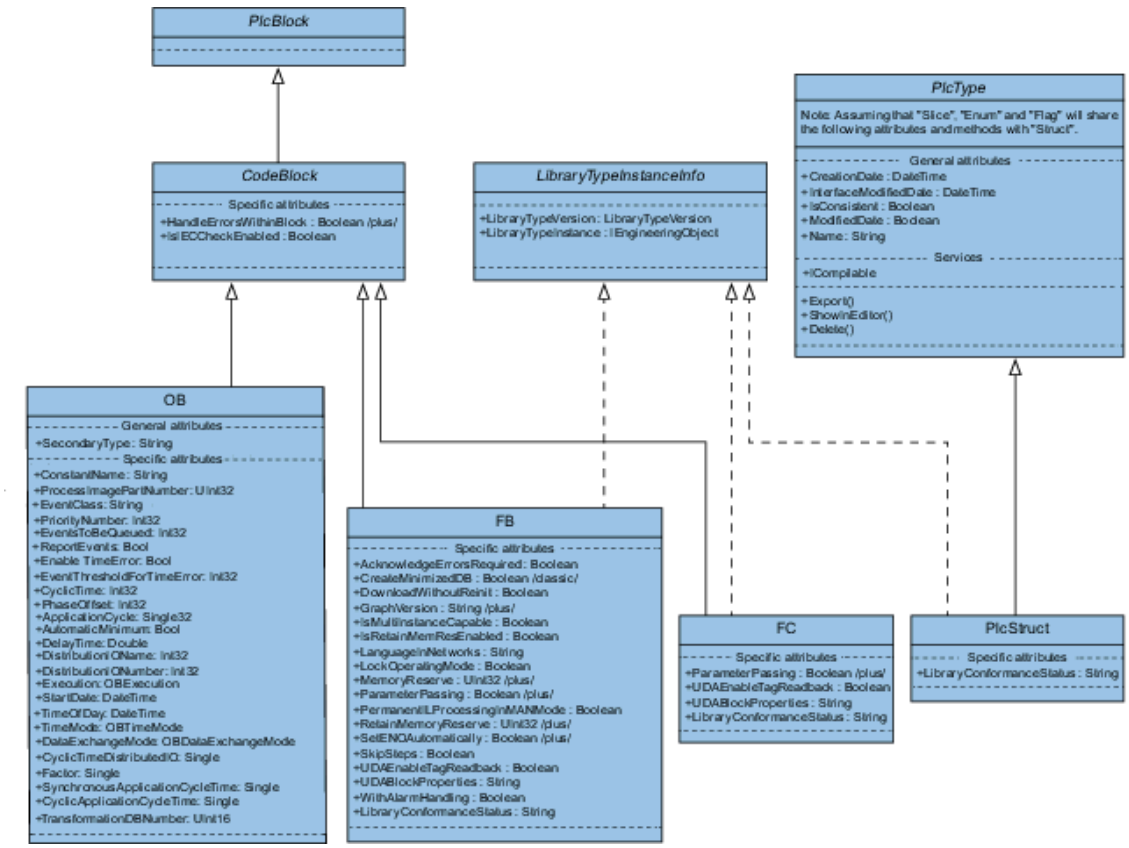
Data


PlcBlock
General attributes
+AutoNumber : Boolean
+CodeModifiedDate : DateTime
+CompileDate : DateTime
+CreationDate : DateTime
+HeaderAuthor : String
+HeaderFamily : String
+HeaderName : String
+HeaderVersion : String
+InterfaceModifiedDate : DateTime
+IsConsistent : Boolean
+IsKnowHowProtected : Boolean
+MemoryLayout : MemoryLayout
+ModifiedDate : DateTime
+Name : String
+Number : Int32
+ParameterModified: DateTime
+ProgrammingLanguage : ProgrammingLanguage
+StructureModified : DateTime
Services
+ICompilable
+Export()
+ShowInEditor()
+Delete()
DataBlock
Specific attributes
+IsOnlyStoredInLoadMemory : Boolean
+IsWriteProtectedInAS : Boolean
ArrayDB
Specific attributes
+ArrayDataType : String
+ArrayLimitUpperBound : Int32
GlobalDB
Specific attributes
+DownloadWithoutReinit : Boolean
+IsPLCDB : Boolean
+IsRetainMemResEnabled : Boolean
+RetainMemoryReserve : UInt32 /plus/
InstanceDB
(Note: The differences between the specific attributes of IDBofFB and IDBofUDT will be handled by
General attributes
+InstanceOfName : String
+OfSystemLibElement : String
+OfSystemLibVersion : String
Specific attributes
+DownloadWithoutReinit : Boolean
+InstanceOfNumber : Int32
+InstanceOfType : BlockType
+IsRetainMemResEnabled : Boolean
+MemoryReserve : UInt32 /plus/
+RetainMemoryReserve : UInt32 /plus/
+UDAEnableTagReadback : Boolean
+UDABlockProperties : String



Representation of block diagram

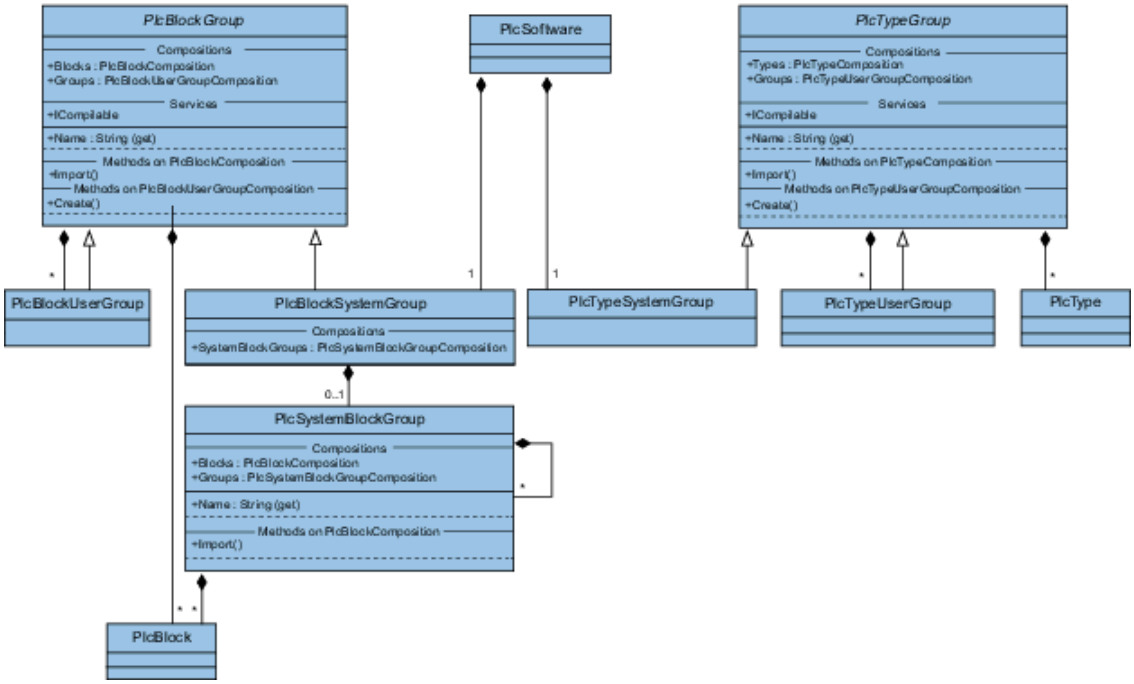
Code and Type



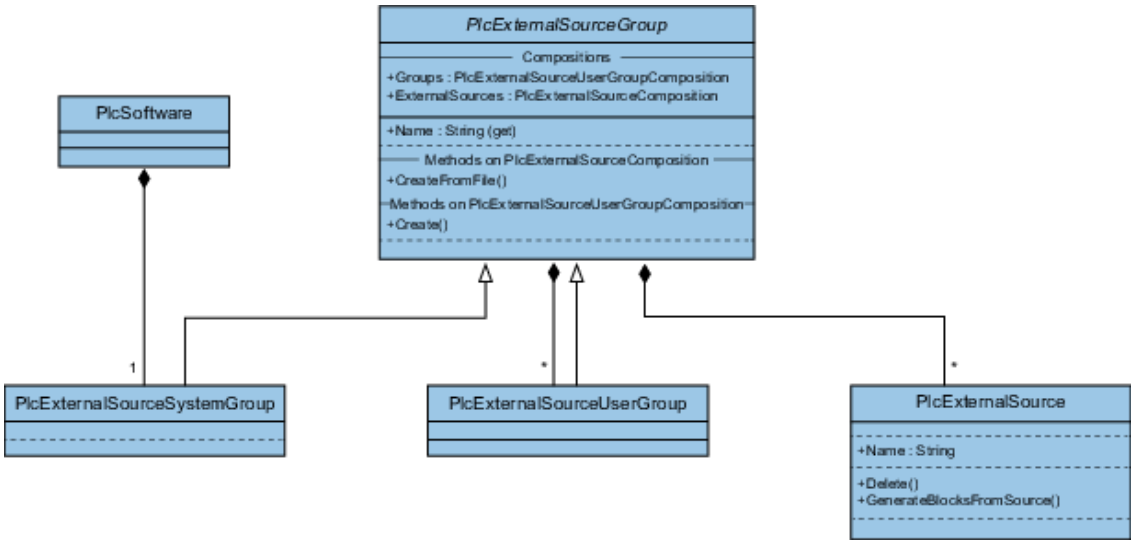
Representation of groups for blocks and types in the TIA Portal Openness API

The two high level groups "PlcBlocks" ("Program blocks" in the GUI of TIA Portal) and "PlcTypes" ("Plc data types" in the GUI of TIA Portal) contain blocks and type definitions. These groups provide the import and the compile functions for blocks. Due to the fact that most of the methods of functionalities of the groups are only achievable via collections, there is an "embedded" or "compacted" representation of the collections and their methods at the "host" classes.

Blocks and Types



External Sources



See also

TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.4.1.3 Hierarchy of hardware objects of the object model

Relation between the visible elements in the TIA Portal and the modeled elements in the object model

Hardware object	Explanation
Device (Device)	The container object for a central or distributed configuration.
Device item (DeviceItem)	Each device item object has a container object. The logical relation is "Items".

To understand the hierarchy relationship of hardware objects between what is displayed in the TIA Portal view and what is modeled in the TIA Portal Openness, it is necessary to consider some aspects of the hierarchy of hardware objects.

Example: A HardwareObject has the items link. The Device extends the HardwareObject and has the UnpluggedItems link. The Device is the container object for a station.

The DeviceItem extends the HardwareObject and has the Container link. Every plugged DeviceItem object has got a container object which is a HardwareObject. The reverse relation is Items.

An unplugged DeviceItem object that does not have a container is found via the link UnpluggedItems of the Device.

The container relation is comparable to the relation of the modules for the device item objects.

Example: A device includes one or more racks. A rack includes modules. A module includes submodules.

This is the relation similar to the representation in the network view and device view of the TIA Portal. The "PositionNumber" attribute of a device item is unique in the items area, within a container.

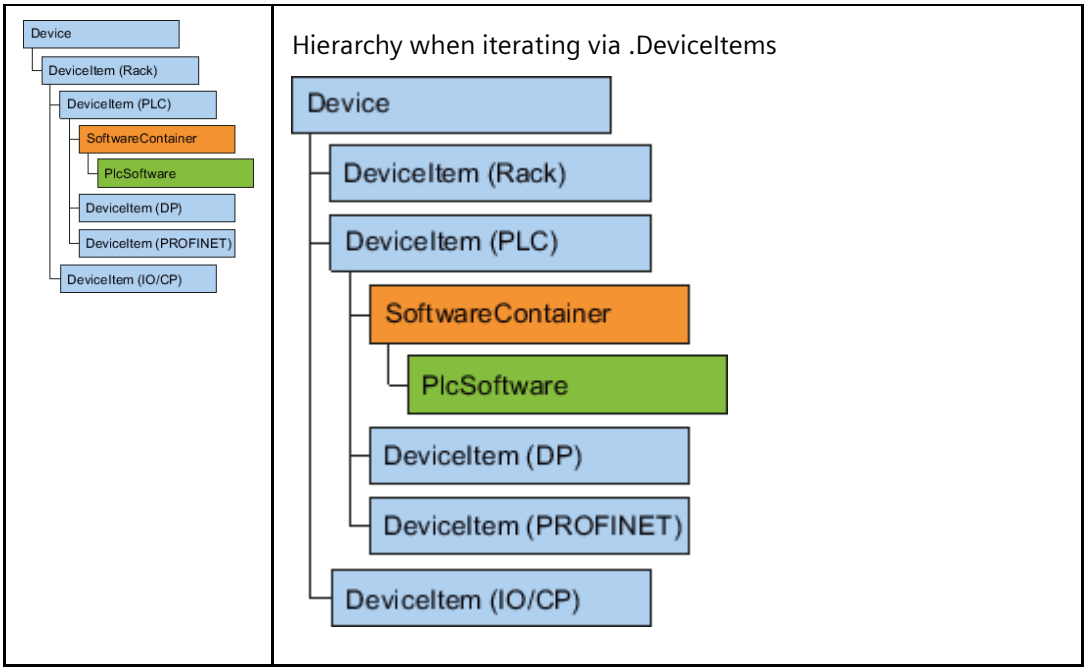
The parent-child relation between device item objects is a purely logical relation in the object model. A child cannot exist without its parents.

- If a submodule is modeled as part of a module (child), the submodule cannot be removed without the module.
- A module or submodule that can be added and removed is a child of the device object

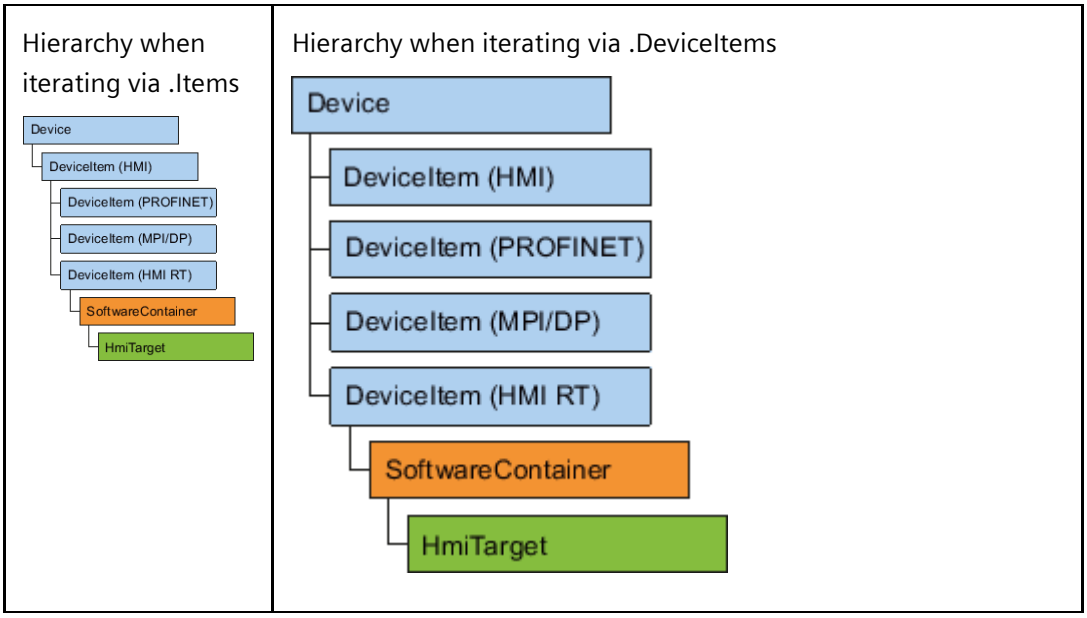
Hierarchy relationships of PLC devices

The diagrams below show the hierarchy relationship between devices, and device items of PLC and HMI devices.

Hierarchy when iterating via .Items	



Hierarchy relationships of HMI devices



See also

TIA Portal Openness object model
Blocks and types of the TIA Portal Openness object model

1.4.1.4 Object list

Introduction

The following tables show the available objects up to and including Runtime Advanced, and indicate whether these objects are supported by TIA Portal Openness.

Neither Runtime Professional nor device proxy files are supported by TIA Portal Openness in WinCC.

Objects

You can control the following project data depending on which HMI device you are using:

Screens

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Screen	Yes	Yes	Yes	Yes
Global screen	Yes	Yes	Yes	Yes
Templates	Yes	Yes	Yes	Yes
Permanent area	No	Yes	Yes	Yes
Pop-up screen	No	Yes	Yes	Yes
Slide-in screen	No	Yes	Yes	Yes

Screen objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Line	Yes	Yes	Yes	Yes
Polyline	No	Yes	Yes	Yes
Polygon	No	Yes	Yes	Yes
Ellipse	Yes	Yes	Yes	Yes
Ellipse segment	No	No	No	No
Circle segment	No	No	No	No
Elliptical arc	No	No	No	No

Camera view	No	No	No	No
Circular arc	No	No	No	No
Circle	Yes	Yes	Yes	Yes
PDF view	No	No	No	No
Rectangle	Yes	Yes	Yes	Yes
Connector	No	No	No	No
Text field	Yes	Yes	Yes	Yes
Graphic view	Yes	Yes	Yes	Yes
Pipe	No	No	No	No
Double T-piece	No	No	No	No
T-piece	No	No	No	No
Pipe bends	No	No	No	No
I/O field	Yes	Yes	Yes	Yes
Date/time field	Yes	Yes	Yes	Yes
Graphic I/O field	Yes	Yes	Yes	Yes
Editable text field	No	No	No	No
List box	No	No	No	No
Combo box	No	No	No	No
Button	Yes	Yes	Yes	Yes
Round button	No	No	No	No
Illuminated button	No	No	Yes	No

Switch	Yes	Yes	Yes	Yes
Symbolic I/O field	Yes	Yes	Yes	Yes
Key-operated switch	No	No	Yes	No
Bar	Yes	Yes	Yes	Yes
Symbol library	No	Yes	Yes	Yes
Slider	No	Yes	Yes	Yes
Scroll bar	No	No	No	No
Check box	No	No	No	No
Option buttons	No	No	No	No
Gauge	No	Yes	Yes	Yes
Clock	No	Yes	Yes	Yes
Memory space view	No	No	No	No
Function keys	Yes	Yes	Yes	Yes
Faceplate instances	No	Yes	Yes	Yes
Screen window	No	No	No	No
User view	Yes	Yes	Yes	Yes
HTML Browser	No	No	No	No
Print job/script diagnostics	No	No	No	No
Recipe view	No	No	No	No
Alarm view	No	No	No	No
Alarm indicator	No	No	No	No

Alarm window	No	No	No	No
Criteria analysis view	No	Yes [1)]	Yes	Yes
ProDiag overview	No	Yes [1)]	Yes	Yes
GRAPH overview	No	Yes [1)]	Yes	Yes
PLC code view	No	Yes [1)]	Yes	Yes
f(x) trend view	No	No	No	No
f(t) trend view	No	No	No	No
Table view	No	No	No	No
Value table	No	No	No	No
Media Player	No	No	No	No
Channel diagnostics	No	No	No	No
WLAN reception	No	No	No	No
Zone name	No	No	No	No
Zone signal	No	No	No	No
Effective range name	No	No	No	No
Effective range name (RFID)	No	No	No	No
Effective range signal	No	No	No	No
Charge condition	No	No	No	No
Handwheel	No	No	Yes	No
Help indicator	No	No	No	No
Sm@rtClient view	No	No	No	No

Status/Force	No	No	No	No
System diagnostic view	No	No	No	No
System diagnostic window	No	No	No	No

1) Only Mobile Panels with the device version greater than 12.0.0.0 support this screen object

Dynamic

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Display	Yes	Yes	Yes	Yes
Operability	No	Yes	Yes	Yes
Visibility	Yes	Yes	Yes	Yes
Movements	Yes	Yes	Yes	Yes

Additional objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Groups	Yes	Yes	Yes	Yes
Soft keys	Yes	Yes	Yes	Yes
Cycles	Yes	Yes	Yes	Yes
VB scripts	No	Yes	Yes	Yes
Function lists	Yes	Yes	Yes	Yes
Project graphics	Yes	Yes	Yes	Yes

Tags

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced

Multiplex tags	Yes	Yes	Yes	Yes
Arrays	Yes	Yes	Yes	Yes
User data types	Yes	Yes	Yes	Yes
Internal	No	Yes	Yes	Yes
Points of use of elementary data types	Yes	Yes	Yes	Yes
Points of use of user data types	Yes	Yes	Yes	Yes
Points of use of arrays	Yes	Yes	Yes	Yes

TIA Portal Openness also supports all value ranges which are supported by the communication drivers.

Connections

TIA Portal Openness supports non-integrated connections that are also supported by the respective HMI devices. You can find additional information in the online help for the TIA Portal under "Process visualization > Controller communication > Device-dependent".

Lists

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Text lists	Yes	Yes	Yes	Yes
Graphic lists	Yes	Yes	Yes	Yes

Texts

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multilingual texts	Yes	Yes	Yes	Yes
Formatted texts and their instances	No	Yes	Yes	Yes

1.4.2 General functions

1.4.2.1 Introduction

Overview

TIA Portal Openness supports a selection of functions for defined tasks that you can call outside the TIA Portal by means of the TIA Portal Openness API.

Note

If a previous version of TIA Portal Openness is already installed, the current version will be installed side by side.

You are provided with an overview of the typical programming steps in the sections below. You can learn how the individual code sections interact and how to integrate the respective functions into a complete program. You also get an overview of the code components that have to be adapted for each task.

Example program

The individual programming steps are explained using the "Creating API access in a console application" function as an example. You integrate the provided functions in this program code and adapt the respective code components for this task.

Functions

The section below lists the functions for defined tasks that you can call with TIA Portal Openness outside the TIA Portal.

See also

Applications
Object list

1.4.2.2 Information about installed TIA Portal Openness versions

Requirement

- TIA Portal Openness and TIA Portal are installed

Introduction

Starting from TIA Portal Openness V14 each installed version has a registry key that contains information about the version. This enables an automatic generation of app.config files for each installed version of TIA Portal Openness.

The registry keys can be located under the following path:

```
HKEY_LOCAL_MACHINE\Software\Siemens\Automation\Openness\xx.x\PublicAPI
```

Note

The version number in this path, is always the number of the currently installed version of TIA Portal. If there are multiple side-by-side installations there are multiple sets of entries for TIA Portal Openness in the registry.

There is a single key for each version of TIA Portal Openness. The names of the Versions will be the same as in the assembly described, for example, the registry entries for TIA Portal Openness:

```
Copies the following program code to clipboard.

[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.x\PublicAPI\
"Siemens.Engineering"="C:\Program Files\Siemens\Automation\Portal Vxx\Pu
"Siemens.Engineering.Hmi"="C:\Program Files\Siemens\Automation\Portal Vx
"EngineeringVersion"="xx"
"AssemblyVersion"="xx.x.x.x"
```

Note

If you want to generate an app.config file you can get the path of the Siemens.Engineering.dll, the Siemens.Engineering.Hmi.dll and the public key token from the registry key.

Openness overview with toolkit reference link

Openness Explorer: https://support.industry.siemens.com/cs/us/en/view/109760816
Demo application: https://support.industry.siemens.com/cs/us/en/view/108716692
Openness Scripter: https://support.industry.siemens.com/cs/us/en/view/109742322

1.4.2.3 TIA Portal Openness IntelliSense support

Application

The Intellisense support of TIA Portal Openness helps you at available attributes or methods via tooltip information. It could contain information about the number, names and types of the required parameters. At the following example the bold parameter in the first line indicates the next parameter that is required as you type the function.

```
project = tiaPortal.Projects.OpenWithUpgrade(projectInfo);
```

Project ProjectComposition.OpenWithUpgrade(FileInfo path)
Open Action with project update is necessary

You can manually invoke Parameter Info by clicking Edit IntelliSense/Parameter Info, typing CTRL+SHIFT+SPACE, or clicking the Parameter Info button on the editor toolbar.

1.4.2.4 Standard libraries

Insert the following namespace statements at the beginning of the relevant code example to ensure that the code examples work:



```
Copies the following program code to clipboard.
using System;
using System.Collections.Generic;
using System.IO;
using Siemens.Engineering;
using Siemens.Engineering.Cax;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Compare;
using Siemens.Engineering.Download;
using Siemens.Engineering.Hmi;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.RuntimeScripting;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Extensions;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.HW.Utilities;
using Siemens.Engineering.Library;
using Siemens.Engineering.Library.MasterCopies;
using Siemens.Engineering.Library.Types;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
```

```
using Siemens.Engineering.SW.TechnologicalObjects;
using Siemens.Engineering.SW.TechnologicalObjects.Motion;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Upload;
```

1.4.2.5 Use of the code examples

Structure of the code-snippets

Each code-snippet in this documentation is implemented as a function without return value with an object reference as transfer parameter. Disposing of objects is omitted for the sake of readability. Objects of the TIA Portal are addressed by their name using the `Find` method.



```
Copies the following program code to clipboard.
//Deletes a single screen from a user folder or a system folder
private static void DeleteScreenFromFolder(HmiTarget hmiTarget)
{
    //The screen "MyScreen" will be deleted if it is existing in the folder "myScreenFolder".
    //If "myScreen" is stored in a subfolder of "myScreenFolder" it will not be deleted.
    string screenName = "MyScreen";
    ScreenUserFolder folder = hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
}
```

You need the following to execute this code-snippet:

- A WinCC project with an HMI device that includes a group with at least one screen.
- A function that instantiates the HMI device.

Note

When you specify directory paths, use the absolute directory path, for example, "C:/path/file.txt".

Relative directory paths are only allowed in the XML files for import and export, for example, "file.txt" or "C:/path01/.../path02/file.txt".

Example for execution of the code-snippet

Use the following example to execute the code-snippet "DeleteScreenFromFolder" as part of the "Hello TIA" example program:

```
Copies the following program code to clipboard.
//In the sample program "Hello TIA" replace the function call
//"IterateThroughDevices(project)" by the following functions calls:
    HmiTarget hmiTarget = GetTheFirstHmiTarget(project);
    DeleteScreenFromFolder(hmiTarget);
//Put the following function definitions before or after the
//function definition of "private static void IterateThroughDevices(Project project)":
private static HmiTarget GetTheFirstHmiTarget(Project project)
{
    if (project == null)
    {
        Console.WriteLine("Project cannot be null");
        throw new ArgumentNullException("project");
    }
    foreach (Device device in project.Devices)
    //This example looks for devices located directly in the project.
    //Devices which are stored in a subfolder of the project will not be affected by this example.
    {
        foreach (DeviceItem deviceItem in device.DeviceItems)
```

```
        {
            DeviceItem deviceItemToGetService = deviceItem as
DeviceItem;
            SoftwareContainer container =
deviceItemToGetService.GetService<SoftwareContainer>();
            if (container != null)
            {
                HmiTarget hmi = container.Software as
HmiTarget;
                if (hmi != null)
                {
                    return hmi;
                }
            }
        }
    }
    return null;
}
//Deletes a single screen from a user folder or a system
folder
private static void DeleteScreenFromFolder(HmiTarget
hmiTarget)
{
    string screenName = "MyScreen";
    ScreenUserFolder folder =
hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
}
```

1.4.2.6 Example program

Application example: Creating API access in an application

The complete program code of the application example is shown below. The typical programming steps are explained next based on this example.

Note

The application example requires an application configuration file.



```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
namespace HelloTIA
```

```
{
    internal class Program
    {
        private static void Main(string[] args)
        {
            RunTiaPortal();
        }
        private static void RunTiaPortal()
        {
            Console.WriteLine("Starting TIA Portal");
            using (TiaPortal tiaPortal = new
TiaPortal(TiaPortalMode.WithUserInterface))
            {
                Console.WriteLine("TIA Portal has started");
                ProjectComposition projects =
tiaPortal.Projects;
                Console.WriteLine("Opening Project...");
                // please adapt the path and the extension
apx to the installed version of TIA Portal
                FileInfo projectPath = new
FileInfo("C:\\Demo\\AnyCompanyProject.apx"); //edit the path
according to your project
                Project project = null;
                try
                {
                    project =
projects.OpenWithUpgrade(projectPath);
                }
                catch (Exception)
                {
                    Console.WriteLine(String.Format("Could
not open project {0}", projectPath.FullName));
```

```
                    Console.WriteLine("Demo complete hit
enter to exit");
                    Console.ReadLine();
                    return;
                }
                Console.WriteLine(String.Format("Project {0}
is open", project.Path.FullName));
                IterateThroughDevices(project);
                project.Close();
                Console.WriteLine("Demo complete hit enter to
exit");
                Console.ReadLine();
            }
        }
        private static void IterateThroughDevices(Project
project)
        {
            if (project == null)
            {
                Console.WriteLine("Project cannot be null");
                return;
            }
            Console.WriteLine(String.Format("Iterate through
{0} device(s)", project.Devices.Count));
            foreach (Device device in project.Devices)
            {
                Console.WriteLine(String.Format("Device: \"
{0}\".", device.Name));
            }
            Console.WriteLine();
        }
    }
}
```

Procedure in steps

1. Make the TIA Portal known in the development environment

In your development environment, create a reference to all "dll files" in the "C:\Program Files\Siemens\Automation\PortalV..\PublicAPI\V.." directory.

The following provides a description of this process using the "Siemens.Engineering.dll" file as an example.

The "Siemens.Engineering.dll" file is available in the directory "C:\Program Files\Siemens\Automation\PortalV..\PublicAPI\V..". Create a reference to the "Siemens.Engineering.dll" file in your development environment.

Note

Ensure that parameter "CopyLocal" is assigned the value "False" in the reference attributes.

2. Publish the name space for the TIA Portal

Add the following code:

```
Copies the following program code to clipboard.
using Siemens.Engineering;
```

3. Publish and start the TIA Portal

In order to publish and start the TIA Portal, insert the following code:

```
Copies the following program code to clipboard.
using (TiaPortal tiaPortal = new TiaPortal())
{
    // Add your code here
}
```

4. Open project

You can use the following code, for example, to open a project:

```
Copies the following program code to clipboard.
ProjectComposition projects = tiaPortal.Projects;
Console.WriteLine("Opening Project...");
//please adapt the path and the extension apx to the
installed version of TIA Portal
```

```
FileInfo projectPath = new
FileInfo("C:\Demo\AnyCompanyProject.apx");
Project project = null;
try
{
    project = projects.Open(projectPath);
}
catch (Exception)
{
    Console.WriteLine(String.Format("Could not open project
{0}", projectPath.FullName));
    Console.WriteLine("Demo complete hit enter to exit");
    Console.ReadLine();
    return;
}
Console.WriteLine(String.Format("Project {0} is open",
project.Path.FullName));
```

5. Enumerate devices of a project

Insert the following code to enumerate all devices of the project:



```
Copies the following program code to clipboard.
static private void IterateThroughDevices(Project project)
    {
        if (project == null)
        {
            Console.WriteLine("Project cannot be null");
            return;
        }

        Console.WriteLine();
        Console.WriteLine(String.Format("Iterate through {0}
device(s)", project.Devices.Count));
        foreach (Device device in project.Devices)
```

```
        {
            Console.WriteLine(String.Format("Device: \"
{0}\".", device.Name));
        }
        Console.WriteLine();
    }
```

6. Save and close the project

Insert the following code to save and close the project:

```
Copies the following program code to clipboard.
project.Save();
project.Close();
```

1.4.2.7 Programming steps

Overview

TIA Portal Openness requires the following programming steps for access by means of the TIA Portal Openness API:

1. Make the TIA Portal known in the development environment
2. Set up program access to the TIA Portal
3. Activate program access to the TIA Portal
4. Publish and start the TIA Portal
5. Open project
6. Execute commands
7. Save and close the project
8. Terminate the connection to the TIA Portal

Note

Permitted strings

Only certain characters are allowed in strings in the TIA Portal. All strings passed to the TIA Portal via the TIA Portal Openness application are subject to these rules. If you pass an invalid character in a parameter, an exception is thrown.

See also

Example program
Use of the code examples

1.4.2.8 Connecting to the TIA Portal

Introduction

You start the TIA Portal with TIA Portal Openness or connect to a TIA Portal already running. When using a TIA Portal Openness application to start the TIA Portal, you specify if the TIA Portal should be started with or without graphical user interface. When you operate the TIA Portal without user interface, the TIA Portal is only started as a process by the operating system. You create several instances of the TIA Portal with a TIA Portal Openness application, if necessary.

Note

If you use TIA Portal Openness with the TIA Portal interface, you cannot use an HMI editor. You can open the "Devices & Networks" editors or the programming editor manually or with TIA Portal Openness API.

You have the following options to start the TIA Portal with a TIA Portal Openness application:

- Use an application configuration file (recommended in most use cases).
- Use the "AssemblyResolve" method (recommended when you use copy deploy etc.).
- Copy the Siemens.Engineering.dll in the TIA Portal Openness application directory.

Note

It is recommended to load the Siemens.Engineering.dll by using the app.config file. By using this method the strong names are considered and malicious modifications to the engineering.dll will result in a loading error. By using the AssemblyResolve method this can't be detected.

Starting the TIA Portal with an application configuration file

Reference all required program libraries in the application configuration file. You distribute the application configuration file together with the TIA Portal Openness application.

Store the application configuration file "app.config" in the same directory as the TIA Portal Openness application and likewise incorporate this in your application. Check whether the file path in each code matches the TIA Portal installation path.

You can use the following code snippet for the application configuration file:

```
Copies the following program code to clipboard.
<?xml version="1.0"?>
<configuration>
    <runtime>
```

```
        <assemblyBinding xmlns="urn:schemas-microsoft-
com:asm.v1">
            <dependentAssembly>
                <assemblyIdentity name="Siemens.Engineering"
culture="neutral" publicKeyToken="d29ec89bac048f84"/>
                <!-- Edit the following path according to
your installed version of TIA Portal -->
                <codeBase version="xx.x.x.x"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\Siemens.Engineering.dll"/>
            </dependentAssembly>
            <dependentAssembly>
                <assemblyIdentity
name="Siemens.Engineering.Hmi" culture="neutral"
publicKeyToken="d29ec89bac048f84"/>
                <!-- Edit the following path according to
your installed version of TIA Portal -->
                <codeBase version="xx.x.x.x"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\Siemens.Engineering.Hmi.dll"/>
            </dependentAssembly>
        </assemblyBinding>
    </runtime>
</configuration>
```

Use the following program code to open a new TIA Portal Instance by means of the application configuration file:

```
Copies the following program code to clipboard.
//Connect a TIA Portal Openness application via API using
using System;
using System.IO;
using Siemens.Engineering;
namespace UserProgram
{
    internal class MyProgram
```

```
    {
        public static void Main(string[] args)
        {
            // To start TIA Portal with user interface:
            // using (TiaPortal tiaPortal = new
TiaPortal(TiaPortalMode.WithUserInterface))
            //
            // To start TIA Portal without user interface:
            // using (TiaPortal tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface))
            using (TiaPortal tiaPortal = new
TiaPortal(TiaPortalMode.WithUserInterface))
            {
                //begin of code for further implementation
                //...
                //end of code
            }
        }
    }
}
```

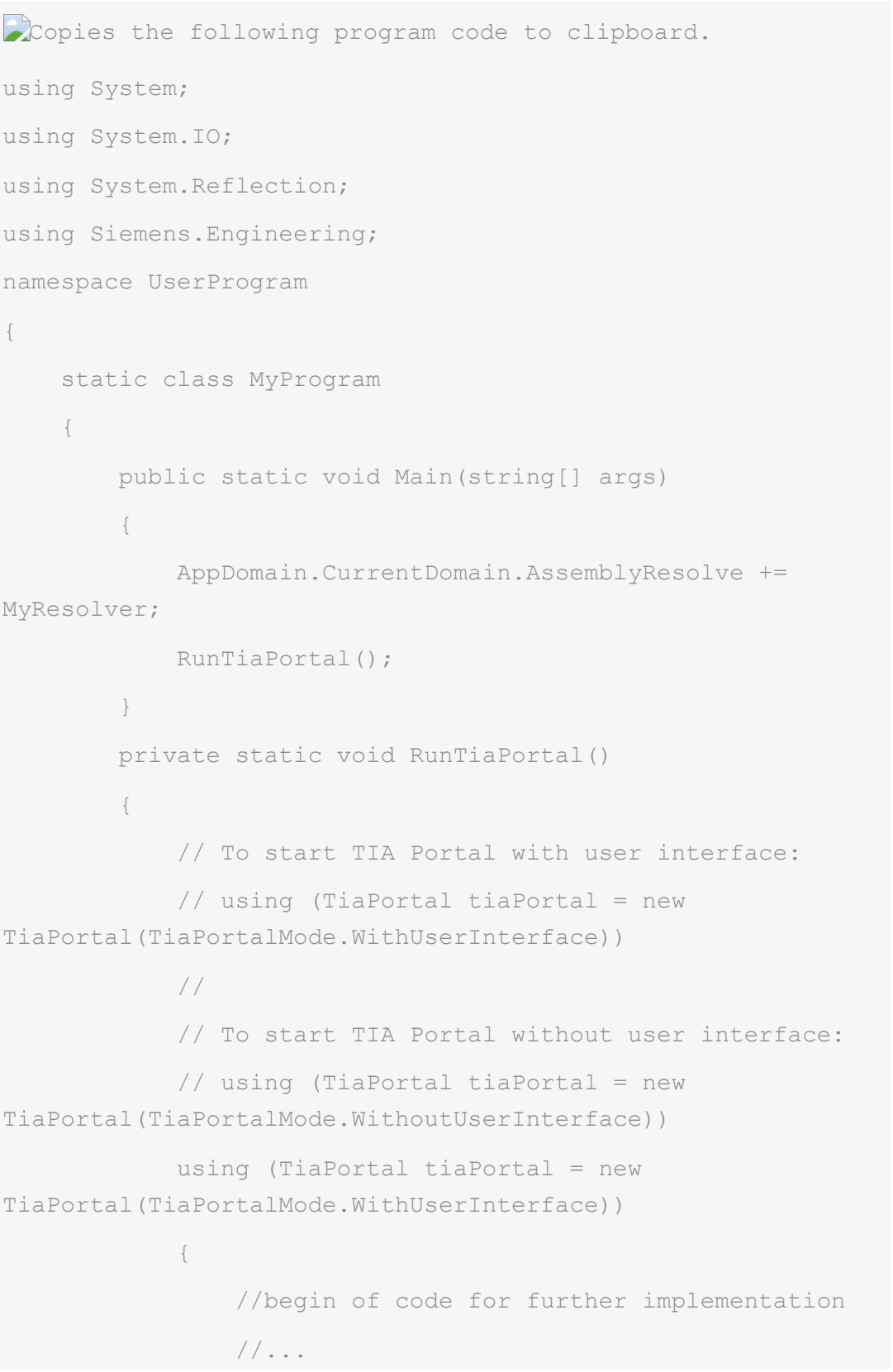
Starting the TIA Portal using the "AssemblyResolve" method

Design the program code of the TIA Portal Openness application in such a way that you register on the event "AssemblyResolve" as early as possible. Encapsulate the access to the TIA Portal in an additional object or method.

Caution must be taken when resolving the engineering assembly using an assembly resolver method. If any types from the engineering assembly are used before the assembly resolver has had run, the program will crash. The reason for this is that the Just-in-time compiler (JIT compiler) doesn't compile methods until it needs to execute them. If engineering assembly types are used in Main, for example, the JIT compiler will attempt to compile Main when the program runs and fail because it doesn't know where to find the engineering assembly. The registration of the assembly resolver in Main doesn't change this. The method needs to run before the assembly resolver is registered, and it needs to be compiled before it can be run. The solution for this problem, is to place the business logic that uses types from the engineering assembly into a separate method that uses only types that the JIT compiler already understands. In the example, a method that returns void and has no parameters and place all business logic inside it is used. When the JIT compiler compiles Main, it will succeed because it knows all the

types in Main. At runtime, when we call RunTiaPortal, the assembly resolver will already be registered, so when the JIT compiler tries to find our business logic types, it will know where to find the engineering assembly.

Use the following program code to open a new TIA Portal Instance.



```
                //end of code
            }
        }
        private static Assembly MyResolver(object sender,
ResolveEventArgs args)
        {
            int index = args.Name.IndexOf(',');
            if (index == -1)
            {
                return null;
            }
            string name = args.Name.Substring(0, index) +
".dll";
            // Edit the following path according to your
installed version of TIA Portal
            string path = Path.Combine(@"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\", name);
            string fullPath = Path.GetFullPath(path);
            if (File.Exists(fullPath))
            {
                return Assembly.LoadFrom(fullPath);
            }
            return null;
        }
    }
}
```

Accessing running instances of the TIA Portal

In order to connect to a running instance of the TIA Portal with a TIA Portal Openness application, start by enumerating the instances of the TIA Portal. You can connect to multiple instances within a Windows session. The running instance can be TIA Portal with or without a started user interface:



```
foreach (TiaPortalProcess tiaPortalProcess in
TiaPortal.GetProcesses())
{
//...
}
```

Note

A TIA Portal instance is not required to enumerate through all running TIA Portal process because GetProcesses is a static method on the class TiaPortal.

If you know the process ID of the instance of the TIA Portal, use this process ID to access the object. TIA Portal requires a certain amount of time to start up before you can connect the TIA Portal Openness application to the TIA Portal.




```
Process process = ...;
Thread.Sleep(TimeNeededToStartUp); // The process may not be
ready to service requests immediately.
TiaPortalProcess tiaPortalProcess =
TiaPortal.GetProcess(process.Id);
```

Note

The process id to which user is connected must be in running state and should have reached where it can respond to requests for process information.

When you connect to a running instance of the TIA Portal, a connection prompt of the TIA Portal Openness firewall appears. The connection prompt offers the following options:

- Allow connection once
- Do not allow connection
- Always allow connections from this application

  See TIA Portal Openness firewall for further information.

  Note

  If the registry prompt is rejected three times, the system throws an exception of the type `EngineeringSecurityException`.

Once you have connected to the process, you can use the following attributes to retrieve information on the instances of the TIA Portal:

Attribute	Information
InstalledSoftware as IList<TiaPortalProduct>	Returns information about the installed products.
Mode as TiaPortalMode	Returns the mode in which the TIA Portal was started (WithoutUserInterface/WithUserInterface).
AttachedSessions as IList<TiaPortalSession>	Returns a list of applications connected to the TIA Portal.
ProjectPath as FileInfo	Returns the file name of the project opened in the TIA Portal, including the folder, for example, "D:\WinCCProjects\ColorMixing\ColorMixing.ap*" If no project is open, a null string is returned.
`ID as int`	Returns the process ID of the TIA Portal instance
`Path as FileInfo`	Returns the path to the TIA Portal executable

Note

TIA Portal Openness does not support a compatibility mode. Each TIA Portal option package or HSP used in the project has to be installed for an Openness application to be able to open or attach to the project.

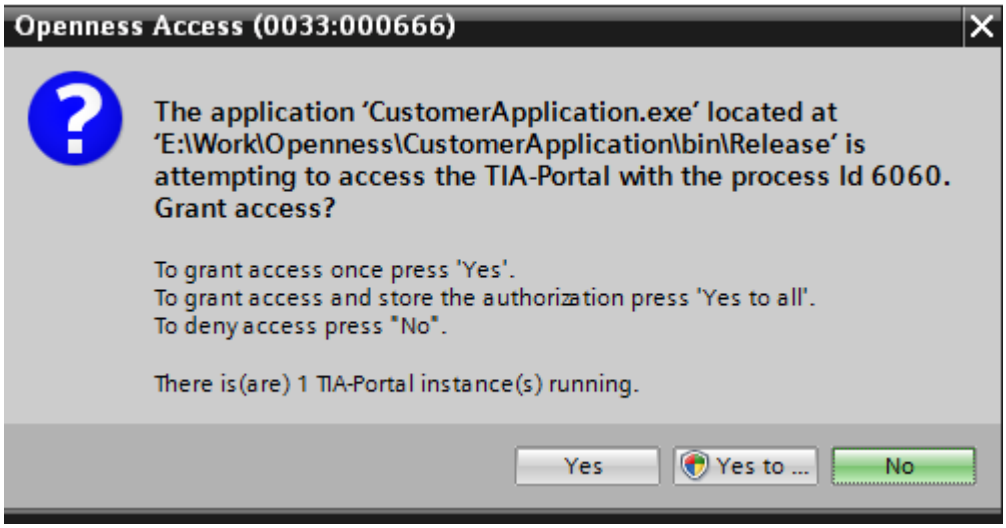
To get the current TIA Portal process from a TIA Portal application by calling GetCurrentProcess() on TIA Portal instance:

```
Copies the following program code to clipboard.
TiaPortal tiaPortal = ...;
TiaPortalProcess tiaPortalProcess =
tiaPortal.GetCurrentProcess();
```

1.4.2.9 TIA Portal Openness firewall

TIA Portal Openness firewall prompt

When you try to connect to a running TIA Portal via TIA Portal Openness, the TIA Portal will prompt you to accept or reject the connection like the following screenshot is showing.



Allow connection to the TIA Portal once

If you just want to connect your TIA Portal Openness application to the TIA Portal once, click "Yes" at the prompt. The next time your TIA Portal Openness application tries to connect the TIA Portal, the prompt will be shown again.

Addition of a whitelist entry by connecting the TIA Portal

To create a whitlist entry for your TIA Portal Openness application follow these steps:

1. Click "Yes to all" at the prompt to display an User Account Control Dialog.
2. Click "Yes" at the User Account Control Dialog to add your application to the whitelist in the windows registry and to attach the application to the TIA Portal.

Addition of a whitelist entry without using the TIA Portal

If you want to add an entry to the whitelist without using TIA Portal you can create a reg file like this:

```
Copies the following program code to clipboard.

Windows Registry Editor Version 5.00

[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.x\Whitelist\

[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.x\Whitelist\

"Path"="E:\\Work\\Openness\\CustomerApplication\\bin\\Release\\CustomerA

"DateModified"="2014/06/10 15:09:44.406"

"FileHash"="0rXRKUCNzMWHOMFrT52OwXzqJef10ran4UykTeBraaY="
```

The following example shows how you can calculate the file hash and last modified date:

```
Copies the following program code to clipboard.
```

```
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
using System.Security.Cryptography;
namespace TIAPortalOpennessFirewall
{
internal class Program
{
private static void Main(string[] args)
{
RunTiaPortal();
```

```
string applicationPath =
@"E:\\Work\\Openness\\CustomerApplication\\bin\\Release\\CustomerApplica
string lastWriteTimeUtcFormatted = String.Empty;
DateTime lastWriteTimeUtc;
HashAlgorithm hashAlgorithm = SHA256.Create();
FileStream stream = File.OpenRead(applicationPath);
byte[] hash = hashAlgorithm.ComputeHash(stream);
// this is how the hash should appear in the .reg file
string convertedHash = Convert.ToBase64String(hash);
lastWriteTimeUtc = fileInfo.LastWriteTimeUtc; // fileInfo not defined
// this is how the last write time should be formatted
lastWriteTimeUtcFormatted = lastWriteTimeUtc.ToString(@"yyyy/MM/dd HH:mm
}
}
private static void RunTiaPortal()
{
Console.WriteLine("Starting TIA Portal");
using (TiaPortal tiaPortal = new TiaPortal(TiaPortalMode.WithUserInterfa
{
Console.WriteLine("TIA Portal has started");
ProjectComposition projects = tiaPortal.Projects;
Console.WriteLine("Opening Project...");
// please adapt the path and the extension apx to the installed version
Portal
FileInfo projectPath = new FileInfo(@"C:\Demo\AnyCompanyProject.apx");
//edit the path according to your project
Project project = null;
try
{
project = projects.OpenWithUpgrade(projectPath);
}
catch (Exception)
```

```
{
Console.WriteLine(String.Format("Could not open project {0}",
projectPath.FullName));
Console.WriteLine("Demo complete hit enter to exit");
Console.ReadLine();
return;
}
Console.WriteLine(String.Format("Project {0} is open",
project.Path.FullName));
IterateThroughDevices(project);
project.Close();
Console.WriteLine("Demo complete hit enter to exit");
Console.ReadLine();
}
}
private static void IterateThroughDevices(Project project)
{
if (project == null)
{
Console.WriteLine("Project cannot be null");
return;
}
Console.WriteLine(String.Format("Iterate through {0}
device(s)", project.Devices.Count));
foreach (Device device in project.Devices)
{
Console.WriteLine(String.Format("Device: \"{0}\".",
device.Name));
}
Console.WriteLine();
}
}
```

```
}
```

1.4.2.10 Event handlers

Event handlers in TIA Portal Openness application

An instance of the TIA Portal provides the following events to which you can react with an event handler in a TIA Portal Openness application. You can access the attributes of notifications and define the responses accordingly.

Event	Response
Disposed	Use this event to respond to the closing of the TIA Portal with a TIA Portal Openness application.
Notification	Use this event to respond to notifications of the TIA Portal with a TIA Portal Openness application. Notifications require only an acknowledgment, e.g. "OK".
Confirmation	Use this event to respond to confirmations of the TIA Portal with a TIA Portal Openness application. Confirmations always require a decision, e.g. "Do you want to save the project?".

Program code

Modify the following program code to register event handlers in a TIA Portal Openness application:

```
 Copies the following program code to clipboard.
//Register event handler for Disposed-Event
    ....
        tiaPortal.Disposed +=TiaPortal_Disposed;
....
private static void TiaPortal_Disposed(object sender,
EventArgs e)
{
    ....
}
//Register event handler for Notification-Event
    ....
```

```
        tiaPortal.Notification += TiaPortal_Notification;
....
private static void TiaPortal_Notification(object sender,
NotificationEventArgs e)
{
    ....
}
//Register event handler for Confirmation-Event
    ....
        tiaPortal.Confirmation += TiaPortal_Confirmation;
....
private static void TiaPortal_Confirmation(object sender,
ConfirmationEventArgs e)
{
    ....
}
```

Attributes of TIA Portal notifications

TIA Portal notifications have the following attributes:

Attribute	Description
`Caption`	Returns the name of the confirmation.
`DetailText`	Returns the detail text of the confirmation.
`Icon`	Returns the icon of the confirmation.
`IsHandled`	Returns the confirmation or specifies if it is still pending.
`Text`	Returns the text of the confirmation.

Attributes of confirmations

Confirmations have the following attributes:

Attribute	Description
`Caption`	Returns the name of the confirmation.
`Choices`	Returns the option to acknowledge the confirmation.
`DetailText`	Returns the detail text of the confirmation.
`Icon`	Returns the icon of the confirmation.
`IsHandled`	Returns the confirmation or specifies if it is still pending.
`Result`	Returns the result of the acknowledgment or specifies it.
`Text`	Returns the text of the confirmation.

See also

Program-controlled acknowledgement of dialogs with system events

1.4.2.11 Program-controlled acknowledgement of dialogs with system events

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- Event handlers are registered.
  See Connecting to the TIA Portal

Application

When you operate the TIA Portal with the user interface, dialogs with system events are displayed for some program sequences. You decide how you want to proceed based on these system events.

When the TIA Portal is accessed with a TIA Portal Openness application, these system events must be acknowledged by means of corresponding ".NET" events.

The permitted confirmations are contained in the `Choices` list:

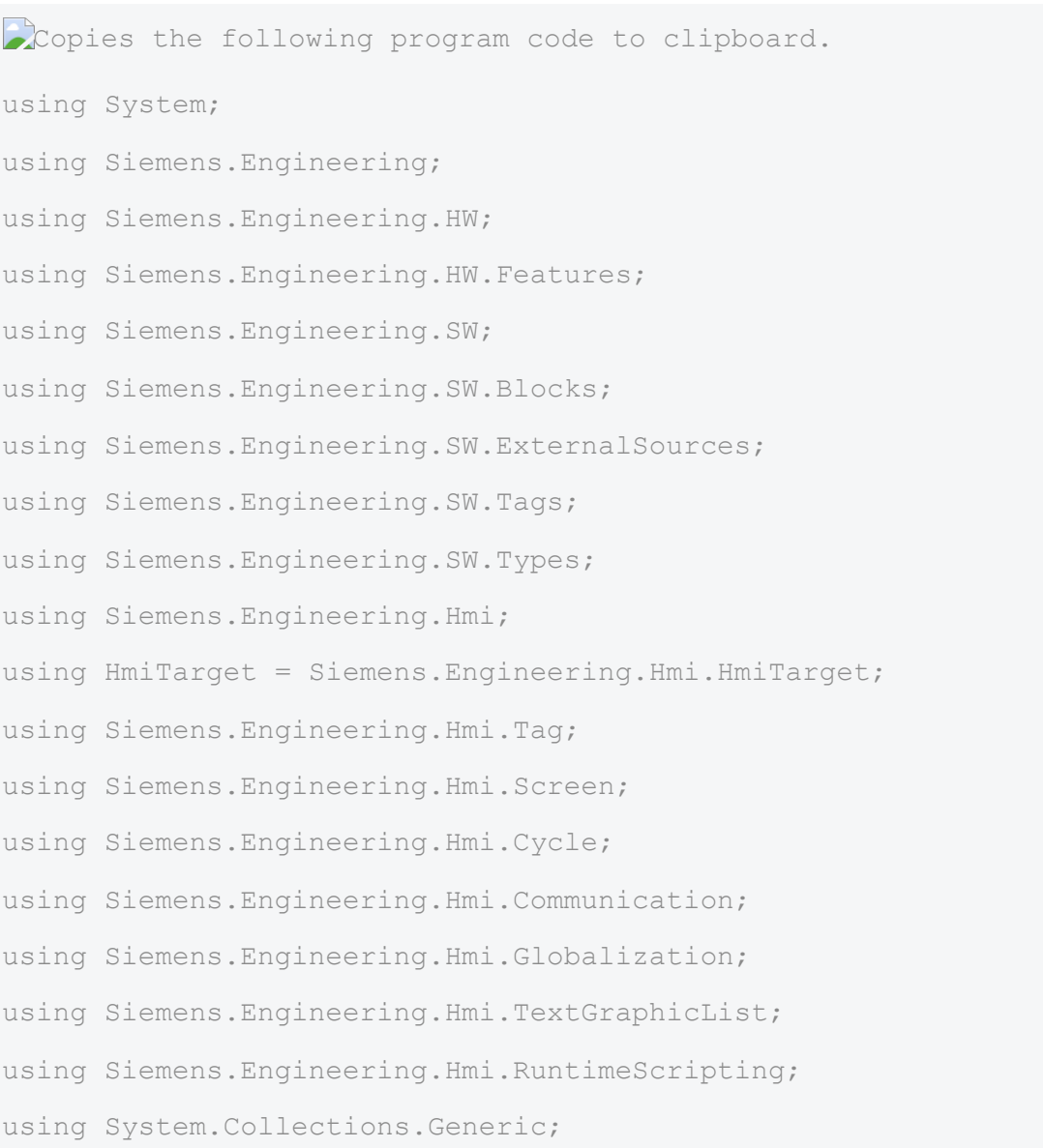
- `Abort`
- `Cancel`

- `No`
- `NoToAll`
- `None`
- `OK`
- `Retry`
- `Yes`
- `YesToAll`

The value of `ConfirmationEventArgs.Result` must be one of the above-mentioned entries. Otherwise, an exception is thrown.

Program code

Modify the following program code to notify the project engineer about executed actions of a TIA Portal Openness application:



```
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
Namespace ConfirmDialogsWithSystemMessagesProgrammatically
{
class internal program()
{
private static void Main(string[] args)
{
// To respond to a confirmation event
using (TiaPortal tiaPortal = new TiaPortal());
{
tiaPortal.Confirmation += TiaPortalConfirmation(object
sender, ConfirmationEventArgs e);
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
}
private void TiaPortalConfirmation(object sender,
ConfirmationEventArgs e)
{
    ...
}
//Handles notifications
using (TiaPortal tiaPortal = new TiaPortal())
{
```

```
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
}
}
}
```

1.4.2.12 Terminating the connection to the TIA Portal

Introduction

If you started the TIA Portal instance without a user interface and if your application is the only TIA Portal Openness client attached to the TIA Portal, you can close the TIA Portal instance with the TIA Portal Openness application. Otherwise, you disconnect the TIA Portal Openness application from the TIA Portal instance.

Use the `IDisposable.Dispose()` method to separate or close the active instance of the TIA Portal.

You can use the `IDisposable.Dispose()` method as follows:

- With a using statement.
- Surround the object description with a try-finally block and call the `IDispose.Dispose()` method within the finally block.

You can no longer access the TIA Portal when you close the active instance of the TIA Portal.

Note

When a configuration engineer closes the TIA Portal instance despite ongoing access of a TIA Portal Openness application, an exception of the class "NonRecoverableException" is thrown in the TIA Portal Openness application on the next API access. You can subscribe to the dispose event to get a call when the TIA Portal is closed.

Program code

Modify the following program code to separate or close the connection to the TIA Portal:

```
Copies the following program code to clipboard.
// Add code to dispose the application if the application is still instantiated
if (tiaPortal != null)
{
    tiaPortal.Dispose();
}
```

See also

Event handlers

1.4.2.13 Diagnostic interfaces on TIA Portal

Application

You can retrieve certain diagnostic information from running instances of TIA Portal via a static method. The diagnostic interface is implemented on the TiaPortalProcess object, which can be retrieved for any currently running instance of the TIA Portal.

The diagnostic interface is not blocking, so you can retrieve the TiaPortalProcess object and access it's members, regardless of if the TIA Portal is busy or not. The diagnostic interface includes the following Members:

Class TiaPortalProcess

Member	Type	Function
AcquisitionTime	DateTime	The time when the TiaPortalProcess object was acquired. Since the TiaPortalProcess object represents a completely static snapshot of the state of the TIA Portal at a given point in time, the information it contains may become outdated.
Attach	TiaPortal	Attaches to the given TiaPortalProcess, it returns a TiaPortal instance.

AttachedSessions	IList<TiaPortalSession>	A collection of all other sessions currently attached to the same TIA Portal. This collection can be empty. Each session is represented by a TiaPortalSession object.
Attaching	EventHandler<AttachingEventArgs>	This event enables an application to approve any attempts to attach to the TIA Portal. When another application attempts to attach to the TIA Portal, the subscribers of this event are notified and given 10 seconds to approve the attachment. If any subscriber ignores this event or does not respond in time, it is understood to be denying the other application permission to attach. Crashed applications, being unable to respond to this event and cannot cause an application to be denied permission to attach.
Dispose	void	Closes the associated TIA Portal instance.
Id	int	The Process ID of the TIA Portal.
InstalledSoftware	IList<TiaPortalProduct>	A collection of all the products currently installed as part of the TIA Portal. Each product is represented by a TiaPortalProduct object, which is described below.
Mode	TiaPortalMode	The mode in which the TIA Portal was started. The current values are

		WithUserInterface and WithoutUserInterface.
Path	FileInfo	The path to the executable of the TIA Portal.
ProjectPath	FileInfo	The path to the project which is currently open in the TIA Portal. If no project is open, this attribute will be null.

Class TiaPortalSession

Member	Type	Function
AccessLevel	TiaPortalAccessLevel	The level access of the session. It is represented as a flags enum, where multiple levels of access are possible. TiaPortalAccessLevel is described in detail below.
AttachTime	DateTime	The time when the connection to the TIA Portal was established.
Id	int	The Id of the current session.
IsActive	bool	Returns "true" if the TIA Portal is currently processing a call from the running session.
Dispose	void	Severs the process's connection to the TIA Portal. This method does not kill the process itself in the way that System.Diagnostics.Process.Kill would do. The application whose connection is terminated will still get a disposed event, but there is no other indication of why the connection was terminated.
ProcessId	int	The process ID of the attached process.

ProcessPath	FileInfo	The path to the executable of the attached process.
TrustAuthority	TiaPortalTrustAuthority	Indicates if the current session was started by a process that was signed, and if it is a TIA Portal Openness certificate or not. TrustAuthority is a flags enum and is described below.
UtilizationTime	TimeSpan	The period of time the process has spent actively using the TIA Portal.Combined with the AttachTime attribute, this could be used to determine usage percentages or similar data.
Version	string	The version of the Siemens.Engineering.dll to which the session is attached to.

Enum TiaPortalAccessLevel

Enum Value	Function
None	This is not a valid value. It is included because TiaPortalAccessLevel is a flags enum which needs an appropriate "zero value" to represent no flags being set, but it will never appear in actual use because no session can be started that has no access.
Published	The session has access to published functionality.
Modify	The session has modify access.

Enum TiaPortalTrustAuthority

Enum Value	Function
None	The main module of the attached process is not signed with a certificate.
Signed	The main module is signed with a certificate, which is not a TIA Portal

	Openness certificate.
Certified	The main module is signed with a TIA Portal Openness certificate.
CertifiedWithExpiration	The main module is signed with a TIA Portal Openness certificate that will become invalid at the end of its lifetime.

Class TiaPortalProduct

Member	Type	Function
Name	string	The name of the product (e.g. STEP 7 Professional).
Options	IList<TiaPortalProduct>	A collection of all optional packages that belong to the connected TIA Portal, represented as TiaPortalProduct objects. If an option package itself has option packages, this nesting could continue.
Version	string	The version string of the product.

The following code snippet provides an example of how to use the diagnostic Interface to query information and of how to use them in your application.



```
Copies the following program code to clipboard.
public void TiaPortalDiagnostics()
{
    IList<TiaPortalProcess> tiaPortalProcesses =
TiaPortal.GetProcesses();
    foreach (TiaPortalProcess tiaPortalProcess in
tiaPortalProcesses)
    {
        Console.WriteLine("Process ID: {0}",
tiaPortalProcess.Id);
```

```
        Console.WriteLine("Timestamp: {0}",
tiaPortalProcess.AcquisitionTime);
        Console.WriteLine("UI Mode: {0}", tiaPortalProcess.Mode);
        //See method body below.
        Console.WriteLine("Installed Software:");
        EnumerateInstalledProducts(tiaPortalProcess.InstalledSoftware);
        Console.WriteLine("Attached Openness Applications:");
        foreach (TiaPortalSession session in
tiaPortalProcess.AttachedSessions)
        {
            Console.WriteLine("Process: {0}", session.ProcessPath);
            Console.WriteLine("Process ID: {0}", session.ProcessId);
            DateTime attachTime = session.AttachTime;
            TimeSpan timeSpentAttached = DateTime.Now - attachTime;
            TimeSpan utilizationTime = session.UtilizationTime;
            long percentageTimeUsed = (utilizationTime.Ticks /
timeSpentAttached.Ticks) * 100;
            Console.WriteLine("AttachTime: {0}", attachTime);
            Console.WriteLine("Utilization Time: {0}",
utilizationTime);
            Console.WriteLine("Time spent attached: {0}",
timeSpentAttached);
            Console.WriteLine("Percentage of attached time spent using
TIA Portal: {0}", percentageTimeUsed);
            Console.WriteLine("AccessLevel: {0}", session.AccessLevel);
            Console.WriteLine("TrustAuthority: {0}",
session.TrustAuthority);
            if ((session.TrustAuthority &
TiaPortalTrustAuthority.Certified) !=
TiaPortalTrustAuthority.Certified)
            {
                Console.WriteLine("TrustAuthority doesn't match
required level, attempting to terminate connection to TIA Portal.");
                session.Dispose();
```

```
            }
        }
    }
}
public void
EnumerateInstalledProducts(IEnumerable<TiaPortalProduct>
products)
{
    foreach (TiaPortalProduct product in products)
    {
        Console.WriteLine("Name: {0}", product.Name);
        Console.WriteLine("Version: {0}", product.Version);
        //recursively enumerate all option packages
        Console.WriteLine("Option Packages \n:");
        EnumerateInstalledProducts(product.Options);
    }
}
```

Security Relevant Information

Because of the fact that no connection to the TIA Portal is needed to use the diagnostics interface, it's possible to write a Windows service that uses the attaching event to check any application attempting to attach to a TIA Portal, e.g. only applications that begin with your company's name are allowed to attach. Another option might be to always grant access, but write information about attaching processes to a log. The following program code is an example event handler to check incoming connections:




```
public void OnAttaching(object sender, AttachingEventArgs e)
{
    string name =
Path.GetFileNameWithoutExtension(e.ProcessPath);
    TiaPortalAccessLevel requestedAccessLevel = e.AccessLevel
& TiaPortalAccessLevel.Published;
    TiaPortalTrustAuthority certificateStatus =
e.TrustAuthority &TiaPortalTrustAuthority.Certified;
```